



Aras Innovator 33

Configurator Services

Programmer's Guide

Document #: D-008096

Last Modified: 8/22/2024

Copyright Information

Copyright © 2024 Aras Corporation. All Rights Reserved.

Aras Corporation
100 Brickstone Square
Suite 100
Andover, MA 01810
Phone: 978-691-8900

E-mail: support@aras.com

Website: <https://www.aras.com/>

Notice of Rights

Copyright © 2024 by Aras Corporation and/or its affiliates. All rights reserved.

This document is protected by U.S. and international copyright laws and conventions. No copyright may be obscured or removed from this document. This document may not be modified or altered, or reproduced or transmitted in any form, without the explicit permission of the copyright holder.

Aras Innovator, Aras, and the Aras Corp "A" logo are registered trademarks of Aras Corporation in the United States and other countries.

All other trademarks referenced herein are the property of their respective owners.

Notice of Liability

THIS DOCUMENT IS PROVIDED FOR INFORMATIONAL PURPOSES ONLY, AND THE CONTENTS HEREOF ARE SUBJECT TO CHANGE WITHOUT NOTICE. THE INFORMATION CONTAINED IN THIS DOCUMENT IS DISTRIBUTED ON AN "AS IS" BASIS, WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE OR A WARRANTY OF NON-INFRINGEMENT. ARAS SHALL HAVE NO LIABILITY TO ANY PERSON OR ENTITY WITH RESPECT TO ANY LOSS OR DAMAGE CAUSED OR ALLEGED TO BE CAUSED DIRECTLY OR INDIRECTLY BY THE INFORMATION CONTAINED IN THIS DOCUMENT OR BY THE SOFTWARE OR HARDWARE PRODUCTS DESCRIBED HEREIN.

Table of Contents

Send Us Your Comments	5
1 Introduction	6
2 Terminology	7
3 Free-Form Boolean Expression Language	8
3.1 Expression Nodes	9
3.1.1 <expression>...</expression>.....	10
3.1.2 <eq>...</eq>.....	10
3.1.3 <variable id="...">.....	10
3.1.4 <named-constant id="...">.....	11
3.1.5 <and>...</and>.....	11
3.1.6 <or>...</or>	11
3.1.7 <not>...</not>.....	12
3.1.8 <implication>...</implication>	14
3.1.9 <exactly-one/>.....	16
3.1.10 <at-most-one />	16
3.1.11 <at-least-one />	16
4 Scope Object Model.....	18
4.1 Scope Object Model Design.....	18
4.2 Configurator API Method Workflow.....	19
4.3 Customizing Scope Builder	19
4.3.1 <i>Implementing the Scope Builder Method</i>	20
4.3.2 <i>Input Item Format</i>	20
4.3.3 <i>Scope Resolver</i>	21
4.3.4 <i>Caching</i>	22
5 API Usage	23
5.1 Brief Description of the API	23
5.2 API Methods.....	24
5.2.1 <i>cfg_GetScopeStructure</i>	24
5.2.2 <i>cfg_GetValidCombinations</i>	29
5.2.3 <i>cfg_ValidateScope</i>	41
5.2.4 <i>cfg_GetIntersectingExpressions</i>	43
5.2.5 <i>cfg_GetConflicts</i>	54
5.2.6 <i>cfg_GetExpressionTruthTable</i>	69
5.2.7 <i>Extra - How To</i>	73
5.3 Influence of Missing Terms on the API results.....	84
6 Configurator Services Interface.....	86
6.1 Getting an IConfiguratorServices interface	86
6.2 Interface description	86

6.2.1	GetScope.....	86
6.2.2	CreateSolver.....	87
6.3	Usage example	88
7	Control API	90
7.1	RuleEditor.....	90
7.1.1	Constructor	90
7.1.2	Public Fields	91
7.1.3	Public Methods	92
7.1.4	Events.....	99
7.1.5	Context Menu Events	100
7.2	VariantsTree.....	101
7.2.1	Constructor	101
7.2.2	Public Fields	102
7.2.3	Public Methods	102
7.2.4	Events.....	104
7.2.5	Context Menu Events	105

Send Us Your Comments

Aras Corporation welcomes your comments and suggestions on the quality and usefulness of this document. Your input is an important part of the information used for future revisions.

- Did you find any errors?
- Is the information clearly presented?
- Do you need more information? If so, where and what level of detail?
- Are the examples correct? Do you need more examples?
- What features did you like most?

If you find any errors or have any other suggestions for improvement, indicate the document title, and the chapter, section, and page number (if available).

You can send comments to us in the following ways:

Email:

TechDocs@aras.com

Subject: Aras Product Documentation

Or,

Postal service:

Aras Corporation
100 Brickstone Square
Suite 100
Andover, MA 01810
Attention: Aras Technical Documentation

If you would like a reply, provide your name, email address, address, and telephone number.

If you have usage issues with the software, visit <https://www.aras.com/support>

1 Introduction

Rapid technological advances and high customer expectations have been driving companies to build highly configurable products with an increasing number of product features and options in all domains of the product.

A product variant is a specific configuration of product for certain option choices, such as size Medium and color Red. The complexity of product variants grows exponentially with the number of features available in the product, and the number of options within each feature. The complexity is multiplied by the time dimension, where new features and options are introduced, or existing features and options are obsoleted. Constraints, driven by Marketing, Engineering, etc. organizations, restrict the set of allowed or compatible option combinations.

Configurator Services, built into the Aras PLM Platform, is the enabler to develop variant management applications to manage product complexity. The configurator uses a rule-based configuration paradigm and has a built-in Boolean Satisfiability (SAT) Solver.

Using Configurator Services, customers can define any data model and business logic to meet business processes. For instance, companies may be managing their products by platforms, product lines, product models, systems/subsystems, or other product groupings. Product features, options and constraints/rules can be defined at all relevant levels for these structures. Configurator Services provides the means to implement server method to translate business data into input data for configurator APIs.

Using Configurator Services, you can:

- ✓ Validate variability definitions
- ✓ Solve variability problems to find a list of valid options
- ✓ Get reasons for invalid options

2 Terminology

Term	Definition
Scope	Class structure to hold data for Configurator Services. The Scope object contains Variables with a list of values that can be assigned to the variable and rules that define constraints within scope variables.
Variable	Variable class is intended to store the variable definition.
Named Constant	Named Constant is intended to store the value definition.
Expression Term	Expression Term class describes connection between variable and value with the <eq> operator. For example: <eq><variable id="color_id" /><named-constant id="red_color_id" /></eq>
Missing Term	An object of Expression Term class that contains at least one ID, which is not in the target Scope object.

3 Free-Form Boolean Expression Language

The Free-Form Boolean expression language enables you to define restrictions and relationships between Variables. It is based on the Aras Markup Language (AML). The following nodes are used in the language:

- expression
- eq
- variable
- named-constant
- and
- or
- not
- implication
- condition
- consequence
- exactly-one
- at-most-one
- at-least-one

Some of these nodes contain required attributes and some of them have a strict structure or required nodes.

When using an expression in AML, it should be represented as a value of a Container Node. This can be achieved in two ways: wrapped with CDATA or encoded.

CDATA Example:

```
<containerNode>
  <![CDATA[<expression>
    <and>
      <eq>
        <variable id="item_id_color" />
        <named-constant id="item_id_red" />
      </eq>
      <eq>
        <variable id="item_id_wheelsize" />
        <named-constant id="item_id_17inch" />
      </eq>
    </and>
  </expression>]]>
</containerNode>
```

Note: There can only be one CDATA node within a Container Node. The CDATA node can only contain one expression node.

Encoded Example:

```
<containerNode>
  &lt;expression&gt;
    &lt;and&gt;
      &lt;eq&gt;
        &lt;variable id="item_id_color" /&gt;
        &lt;named-constant id="item_id_red" /&gt;
      &lt;/eq&gt;
      &lt;eq&gt;
        &lt;variable id="item_id_wheelsize" /&gt;
        &lt;named-constant id="item_id_17inch" /&gt;
      &lt;/eq&gt;
    &lt;/and&gt;
  &lt;/expression&gt;
</containerNode>
```

The following sections describe these nodes and include examples.

3.1 Expression Nodes

Dependencies between expression nodes are listed in the following table.

Tag	Can contain
expression	implication, and, or, not, eq, exactly-one, at-most-one, at-least-one
eq	variable, named-constant
variable	id attribute
named-constant	id attribute
and	implication, and, or, not, eq, exactly-one, at-most-one, at-least-one
or	implication, and, or, not, eq, exactly-one, at-most-one, at-least-one
not	implication, and, or, not, eq, exactly-one, at-most-one, at-least-one
implication	condition, consequence
condition	implication, and, or, not, eq, exactly-one, at-most-one, at-least-one
consequence	implication, and, or, not, eq, exactly-one, at-most-one, at-least-one

Tag	Can contain
exactly-one	eq
at-least-one	eq
at-most-one	eq

3.1.1 <expression>...</expression>

The <expression> node is a root node that represents the expression. Use the <expression> tag to define any expression in Boolean language.

```
<expression>
  <and>
    <eq>
      <variable id="item_id_color" />
      <named-constant id="item_id_red" />
    </eq>
    <eq>
      <variable id="item_id_wheelsize" />
      <named-constant id="item_id_17inch" />
    </eq>
  </and>
</expression>
```

This example represents the Boolean expression for Color = Red AND WheelSize = 17 inch.

3.1.2 <eq>...</eq>

The <eq> node defines equivalence. Equivalence can be done between Variable and NamedConstant. <eq> has a strict content: it must include the node pair <variable> and <named-constant>.

```
<eq>
  <variable id="item_id_color" />
  <named-constant id="item_id_red" />
</eq>
```

This example represents the Boolean expression for Color = Red.

3.1.3 <variable id="..." />

The <variable> node defines a variable element. Use this element to define the first part of an equivalence. The first part of an equivalence should always be the variable. The "id" attribute is required; it defines the unique identifier for the instance.

```
<eq>
  <variable id="item_id_color" />
  <named-constant id="item_id_red" />
</eq>
```

This example represents the Boolean expression for Color = Red where Color is the variable.

3.1.4 <named-constant id="...">

The <named-constant> node defines the namedConstant element. Use this element to define the second part of equivalence. The second part of equivalence should always be namedConstant. The "id" attribute is required; it defines the unique identifier for the instance.

```
<eq>
  <variable id="item_id_color" />
  <named-constant id="item_id_red" />
</eq>
```

This example represents the Boolean expression for Color = Red where Red is the named constant.

3.1.5 <and>...</and>

The <and> node that defines the Boolean operation "and". It can contain an unlimited number of child tags.

Note: It is unnecessary to include the <and></and> node explicitly within the expression node because is already implied.

```
<and>
  <eq>
    <variable id="item_id_color" />
    <named-constant id="item_id_red" />
  </eq>
  <eq>
    <variable id="item_id_color" />
    <named-constant id="item_id_green" />
  </eq>
</and>
```

This example represents the Boolean expression for Color = Red AND Color = Green.

3.1.6 <or>...</or>

The <or> node defines the Boolean operation "or". It can contain an unlimited number of child tags.

```
<or>
  <eq>
    <variable id="item_id_color" />
    <named-constant id="item_id_red" />
  </eq>
```

```
<eq>
    <variable id="item_id_color" />
    <named-constant id="item_id_green" />
</eq>
```

</or>

This example represents the Boolean expression for Color = Red OR Color = Green.

The following is an example of an OR node using an inner AND node:

<or>

```
<eq>
    <variable id="item_id_color" />
    <named-constant id="item_id_red" />
</eq>
```

<and>

```
<eq>
    <variable id="item_id_wheelsize" />
    <named-constant id="item_id_17inch" />
</eq>
```

```
<eq>
    <variable id="item_id_color" />
    <named-constant id="item_id_green" />
</eq>
```

</and>

</or>

This example represents the Boolean expression for Color = Red OR (WheelSize = 17inch AND Color = Green).

3.1.7 <not>...</not>

The <not> node defines the Boolean operation “not”. It can contain an unlimited number of child tags.

<not>

```
<eq>
    <variable id="item_id_color" />
    <named-constant id="item_id_red" />
</eq>
```

</not>

This example represents the Boolean expression for NOT(Color = Red).

<not>

```
<or>
    <eq>
```

```

        <variable id="item_id_wheelsize" />
        <named-constant id="item_id_17inch" />
    </eq>
    <eq>
        <variable id="item_id_color" />
        <named-constant id="item_id_green" />
    </eq>
</or>
</not>

```

This example represents the Boolean expression for NOT(WheelSize = 17 inch OR Color = Green). This is the same as NOT(WheelSize = 17 inch) AND NOT(Color = Green).

Note: When an explicit tag to wrap child tags is not provided, the child tags are wrapped by the *and* operator. It is explained in below example.

The following example has multiple child nodes contained within the `<not>` node:

```

<not>
    <eq>
        <variable id="item_id_wheelsize" />
        <named-constant id="item_id_16inch" />
    </eq>
    <eq>
        <variable id="item_id_color" />
        <named-constant id="item_id_red" />
    </eq>
</not>

```

The above example is equivalent to the case when child nodes are wrapped by `<and>` node as below:

```

<not>
    <and>
        <eq>
            <variable id="item_id_wheelsize" />
            <named-constant id="item_id_16inch" />
        </eq>
        <eq>
            <variable id="item_id_color" />
            <named-constant id="item_id_red" />
        </eq>
    </and>
</not>

```

This example represents the Boolean expression for NOT(WheelSize = 16 inch AND Color = Red). This is the same as NOT(WheelSize = 16 inch) OR NOT(Color = Red).

3.1.8 <implication>...</implication>

The <implication> node defines a Boolean operation "implication" for example "if Color = Red then WheelSize = 17inch". This node should always contain the <condition> and <consequence> child nodes. The order of the child nodes is important: the <condition> node must always precede the <consequence> node. These nodes should not be left empty.

```
<implication>
  <condition>
    <eq>
      <variable id="item_id_color" />
      <named-constant id="item_id_red" />
    </eq>
  </condition>
  <consequence>
    <eq>
      <variable id="item_id_wheelsize" />
      <named-constant id="item_id_17inch" />
    </eq>
  </consequence>
</implication>
```

This example represents the Boolean expression for IF Color = Red THEN WheelSize = 17inch.

Note: When an explicit tag to wrap child tags within the <condition> or <consequence> nodes is not provided, the child tags are wrapped by **and** operator. It is explained in below example.

The following example shows multiple child nodes contained within the <condition> and <consequence> nodes:

```
<implication>
  <condition>
    <eq>
      <variable id="item_id_color" />
      <named-constant id="item_id_red" />
    </eq>
    <eq>
      <variable id="item_id_type" />
      <named-constant id="item_id_mountain" />
    </eq>
  </condition>
  <consequence>
```

```

    <eq>
      <variable id="item_id_wheelsize" />
      <named-constant id="item_id_16inch" />
    </eq>
    <eq>
      <variable id="item_id_framesize" />
      <named-constant id="item_id_large" />
    </eq>
  </consequence>
</implication>

```

The above example is equivalent to the case when they are wrapped by `<and>` node as shown below:

```

<implication>
  <condition>
    <and>
      <eq>
        <variable id="item_id_color" />
        <named-constant id="item_id_red" />
      </eq>
      <eq>
        <variable id="item_id_type" />
        <named-constant id="item_id_mountain" />
      </eq>
    </and>
  </condition>
  <consequence>
    <and>
      <eq>
        <variable id="item_id_wheelsize" />
        <named-constant id="item_id_16inch" />
      </eq>
      <eq>
        <variable id="item_id_framesize" />
        <named-constant id="item_id_large" />
      </eq>
    </and>
  </consequence>
</implication>

```

This example represents the Boolean expression for IF (Color = Red AND Type = Mountain) THEN WheelSize = 16inch AND FrameSize = Large.

3.1.9 <exactly-one/>

The `<exactly-one>` node defines an operation. The operator describes a condition that means “one and only one of the listed equivalencies can be true”. `<exactly-one>` should contain a set of `<eq>` child nodes.

```
<exactly-one>
  <eq>
    <variable id="item_id_color" />
    <named-constant id="item_id_red" />
  </eq>
  <eq>
    <variable id="item_id_color" />
    <named-constant id="item_id_green" />
  </eq>
</exactly-one>
```

This example represents the Boolean expression for EXACTLY-ONE(Color = Red | Color = Green).

3.1.10 <at-most-one />

The `<at-most-one>` node defines an operation. The operator describes a condition that means “either none or just one of the listed equivalencies can be true”. `<at-most-one>` should contain a set of `<eq>` child nodes.

```
<at-most-one>
  <eq>
    <variable id="item_id_color" />
    <named-constant id="item_id_red" />
  </eq>
  <eq>
    <variable id="item_id_color" />
    <named-constant id="item_id_green" />
  </eq>
</at-most-one>
```

This example represents the Boolean expression for AT-MOST-ONE(Color = Red | Color = Green).

3.1.11 <at-least-one />

The `<at-least-one>` node defines an operation. The operator describes a condition that means “at least one of the listed equivalencies should be true”. `<at-least-one>` should contain a set of `<eq>` child nodes:

```
<at-least-one>
  <eq>
    <variable id="item_id_color" />
    <named-constant id="item_id_red" />
  </eq>
  <eq>
```

```
<variable id="item_id_color" />  
  <named-constant id="item_id_green" />  
</eq>  
</at-least-one>
```

This example represents the Boolean expression for AT-LEAST-ONE(Color = Red | Color = Green).

4 Scope Object Model

Configurator Services has an internal Scope object model that is customizable. Configurator Services processing logic is built around this object model.

This section describes the following:

- Scope Object Model
- Configurator API Method Workflow
- Building the Scope Object Model

4.1 Scope Object Model Design

The Scope object contains Variables with a list of values that can be assigned to the variable and rules that define relationships between assigned variables.

The following diagram illustrates the structure of the Scope Object Model:

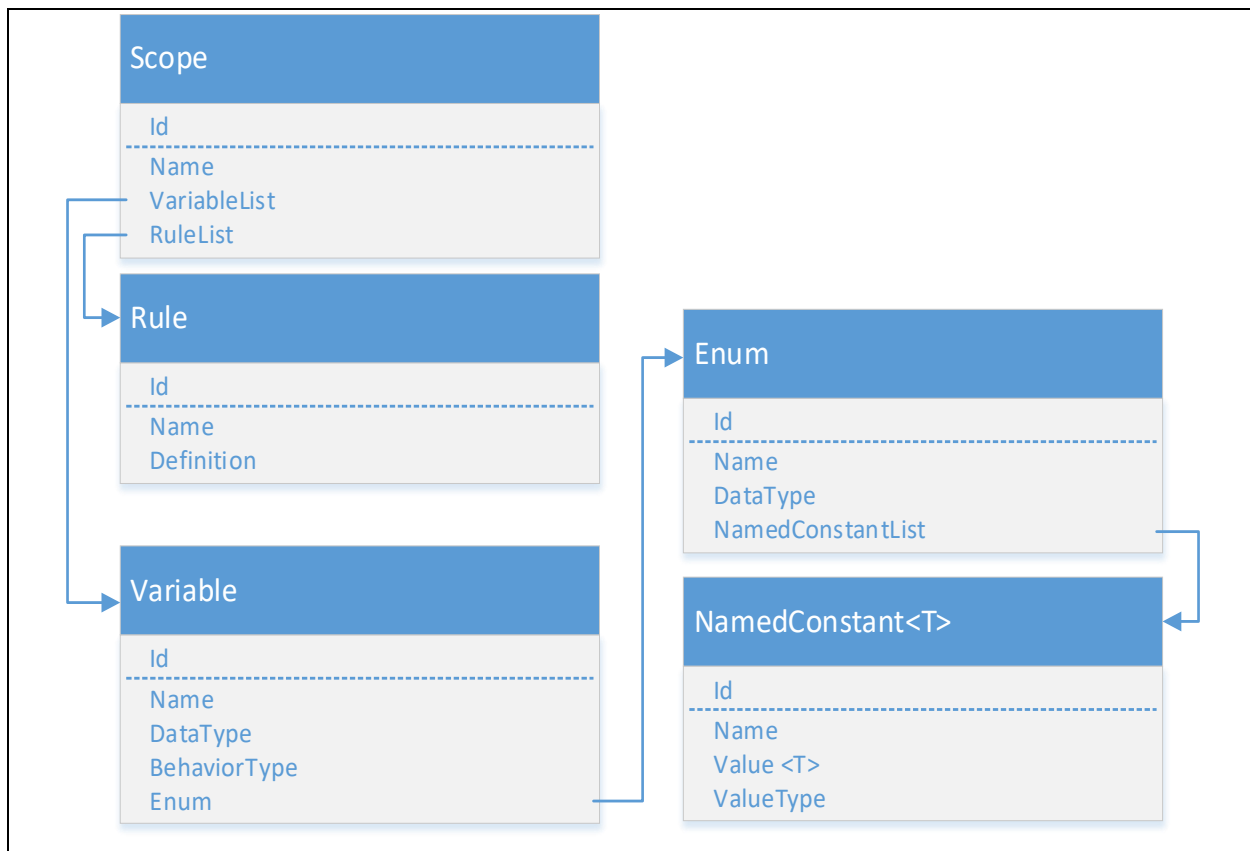


Figure 1.

- Scope is a class that contains two lists: a list of Variables and a list of Rules.
- Variable class stores the variable definition. BehaviorType defines the restriction of the number of values in a variable for solving the scope. The BehaviorType can be exactly-one, at-most-one or at-least-one. If BehaviorType is not specified, the default restriction is exactly-one.

- Named constant is intended to store the value definition. It contains the property name, value, and valuetype.
- Enum is the container for the list of Named constants.
- Rule is a container for an expression. The Rule has both a name and definition where the definition type is ExpressionBase.
- ExpressionBase is an object representation of a Boolean expression.

4.2 Configurator API Method Workflow

The following workflow diagram illustrates the process of collecting data from its source and using this data to perform an API Action.

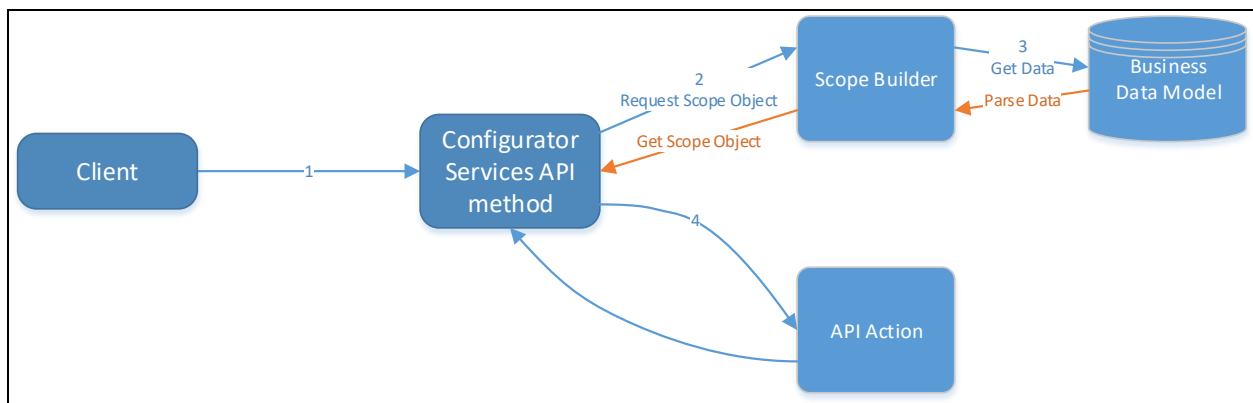


Figure 2.

1. The Client calls the Configurator Services API method.
2. The Configurator Services API method requests the scope builder method to build the Scope object.
3. The scope builder method collects and parses data from the business data source.
4. The Configurator Services API method performs an API action in the context of the built Scope object.

4.3 Customizing Scope Builder

Scope Builder is a server method that uses a predefined method template. The template enables customers to implement a Scope builder method. All of the Configurator Services API methods that perform actions in the context of Scope should be parameterized with the Scope object. The following is an example of the method template:

```
<Item type="Method" action="%action name%" %specific API attributes%>
  <targetScope>
    <Item type="Method" action="%builder method name%" %builder method attributes%>
      %builder method properties%
    </Item>
  </targetScope>
  %specific API properties%
</Item>
```

4.3.1 Implementing the Scope Builder Method

The Scope Builder method uses the CSharp:Aras.Server.Core.Configurator template to define a class that inherits from the base abstract class. The following is the template for the Scope builder method:

```
namespace $(pkgname)
{
    public class $(clsname): ScopeBuilderBase
    {
        $(MethodCode)
    }
}
```

Base abstract class:

```
public abstract class ScopeBuilderBase
{
    public Item ScopeItem { get; set; }
    public IServerConnection IomConnection { get; private set; }
    public void Init(Item scopeItem, IServerConnection iomConnection) {}
    public abstract Scope BuildScope();
    public abstract string[] GetGuidsItemDependsOn();
    public abstract List<string> GetItemTypeNameItemDependsOn();
    public abstract ArrayList GetCustomKey();
}
```

The Scope Builder method is designed to provide a way for constructing a Scope object. The Scope Builder method also provides the built-in possibility to cache the Scope object.

Note: Anyone who implements a scope builder method will be forced to override all abstract methods.

Note: It is recommended to implement methods operating system agnostic for use with Linux and Windows. The primary OS incompatibility considerations for method implementation are path case-sensitivity, path separators, OS-specific line endings and OS-specific code. For more information about cross-platform development, please refer to section 2.3 *Cross-platform development* in *Aras Innovator - Programmer's Guide*.

4.3.2 Input Item Format

The targetScope Property enables you to specify the Scope builder method for Configurator Services API methods using AML syntax.

The syntax is as follows:

```
<Item type="Method" action="%action name%" %specific API attributes%>
  <targetScope>
    <Item type="Method" action="%builder method name%" %builder method attributes%>
      %builder method properties%
    </Item>
  </targetScope>
  %specific API properties%
</Item>
```

Where:

%action name%	Name of the action associated with the Configurator API method.
%specific API attribute%	List of API attributes specifically for a particular method. For example, "fetch" is an API attribute for the cfg_GetValidCombinations and cfg_GetConflicts methods.
%builder method name%	Name of the server method that builds the Scope object model based on the business logic being used.
%builder method attributes%	Attributes that are needed for the builder method to process a business Item.
%builder method properties%	Properties that are needed for the builder method to process the business item.
%specific API properties%	List of the API properties specific to a particular method. For example, the "condition" property is specific to the cfg_GetValidCombinations method.

%specific API attributes% and **%specific API properties%** define the method signature that is specified in the action attribute.

%builder method attributes% and **%builder method properties%** should contain all the information that is necessary for building the Scope object.

Note: Samples are provided for each method in [5 API Usage](#).

4.3.3 Scope Resolver

Scope Resolver is an internal module that is used as part of the Scope Builder process. It is used to parse request AML, call for the Scope builder method, and cache the Scope builder method result. The following description is for information only, to make internal processing clearer:

```
public class ScopeResolverModule
{
    public Scope BuildScope(Item item)
    {
        ScopeBuilderBase builder = GetBuilder(item);
        builder.Init(item, this.serverConnection);
        return this.cache.CacheDriverNotNull(
```

```

        CachableScopeContainer.GetKey(builder.GetCustomKey()), list =>
        CachableScopeContainer.GetInstance(builder, this.cacheItemType)).Scope;
    }
    private ScopeBuilderBase GetBuilder(Item scopeItem) { }
}

```

4.3.4 Caching

The Scope builder approach provides the built-in ability to cache a built Scope object. You need to override the CustomKey function in order to store and retrieve the scope object from the cache. This function should return the same ArrayList for each request to the same Scope object. To store a different cache for different Scope objects, the GetCustomKey function should return the different contents of the ArrayList:

```

public override ArrayList GetCustomKey()
{
    return new ArrayList {
        ScopeItem.getID(),
    };
}

```

The Scope object can be invalidated automatically. You need to create the following collections to make this possible:

- List of ItemType Names. This list contains the name of each Item Type that was used to build the Scope.

```

public override List<string> GetItemTypeNamesItemDependsOn()
{
    return new List<string> {
        "ItemType1",
        "ItemType2",
        "ItemType3",
        "ItemType4"
    };
}

```

Each Item Type in this list should be a polysource item for **ScopeCacheDependency**.

- List of Item IDs. This list contains the ID of each Item that was used to build the Scope.

```

public override string[] GetGuidsItemDependsOn()
{
    return new string[] {
        "item_id_color",
        "item_id_red",
        "item_id_wheelsize",
        "item_id_17inch"
    };
}

```

5 API Usage

The following sections describe how to use the Configurator Services API. This section describes the following topics in detail:

- Configurator Services API methods
- AML Request Syntax
- AML Response Formats
- AML Request/Response examples

Along with AML, Configurator Services API methods can be called using Aras Innovator's RESTful API.

5.1 Brief Description of the API

The Configurator Services API consists of several methods. This section describes the purpose of these methods.

Configurator Services works with any custom business model. The Scope Builder method takes the customer data and translates it into a Scope object that is used by the Configurator Services API. The Scope object is used by the Configurator Services API to solve the appropriate task.

The following API methods use a scope object:

- **cfg_GetScopeStructure.** This method creates a representation of the Scope object: it uses `scope_builder` to build the Scope object, which is then serialized into either AML or JSON. The method can be used to:
 - display a list of available choices
 - get actual data inside implementation of new server methods
 - for debugging purposes
- **cfg_GetValidCombinations.** This method is designed to find a list of valid combinations. Valid Combination is a set of variables with assigned values to them, which leads to valid solution. It can be used to:
 - find all valid combinations
 - find at least one valid combination
 - validate the current scope
 - validate values selected for Variables
 - validate an expression
- **cfg_ValidateScope.** This method is designed to validate the Scope. The method has two validations:
 - if Scope has at least one valid combination
 - if each value can be assigned to a Variable for at least one valid combination

The method can be used to:

- find out if the specified Scope is solvable
- determine if unreachable values exist

- **cfg_GetIntersectingExpressions.** This method is designed to find the intersections of expressions. Expressions are intersecting if the Scope has at least one valid combination with specified expressions applied.
- **cfg_GetConflicts.** This method is designed to find the reason why the current Scope is unsolvable. The method can be used to:
 - get data that describes reasons
 - debug why you have no solutions

5.2 API Methods

5.2.1 cfg_GetScopeStructure

This method retrieves a Scope Structure. To request the Scope Structure, invoke the method with an AML or OData request. The method returns the Scope Structure in AML or JSON respectively.

The output allows customizations to include extended information about Items.

5.2.1.1 AML Request Syntax

```
<Item type="Method" action="cfg_GetScopeStructure">
  <targetScope>
    <Item type="Method" action="%builder method name%" %builder method attributes% >
      %builder method properties%
    </Item>
  </targetScope>
  <output_builder_method>%output_builder_method%</output_builder_method>
</Item>
```

Where:

<targetScope /> %builder method name%	Refer to 4.3.2 Input Item Format . The <targetScope /> node is required. For Missing Terms affect, refer to 5.3 Influence of Missing Terms on the API results .
%builder method attributes%	Builder method attributes.
%builder method properties%	Builder method properties.
%output_builder_method%	Optional parameter to provide the Name of a server method that will override GetScopeStructureOutputBase using the Aras.Server.Core.Configurator.ScopeStructureOutput template. The server method is used to extend responses using custom data. The AML targetScope item is accessible in the output_builder_method.

5.2.1.2 AML Response Format

```
<Item type="Scope" id="ScopeId">
  <name>ScopeName</name>
  <Relationships>
    <Item type="Variable" id="VariableId_1">
      <id>VariableId_1</id>
      <name>VariableName_1</name>
      <Relationships>
        <Item type="NamedConstant" id="NamedConstantId_1">
          <id>NamedConstantId_1</id>
          <name>NamedConstantName_1</name>
        </Item>
        <Item type="NamedConstant" id="NamedConstantId_2">
          <id>NamedConstantId_2</id>
          <name>NamedConstantName_2</name>
        </Item>
      </Relationships>
    </Item>
    ...
    <Item type="Rule" id="RuleId_1">
      <id>RuleId_1</id>
      <name>RuleName_1</name>
      <definition>expression</definition>
    </Item>
    ...
  </Relationships>
</Item>
```

5.2.1.3 OData Request Syntax

```
{
  "targetScope": {
    "Item": {
      "@aras.type": "Method",
      "@aras.action": "%builder method name%",
      "@aras.%builderMethodAttributeName1%": "%attribute 1 value%",
      "@aras.%builderMethodAttributeName2%": "%attribute 2 value%",
      ...
      "%builderMethodPropertyName1%": "%property 1 value%",
      "%builderMethodPropertyName2%": "%property 2 value%"
    }
  },
  "output_builder_method": "%output_builder_method%"
}
```

Note: For more information about using OData protocol, refer to the *Aras Innovator – RESTful API*.

Where:

targetScope %builder method name%	Refer to 4.3.2 Input Item Format . The "targetScope" object is required.
%builderMethodPropertyName#%	A builder method property.
%builderMethodAttributeName#%	A builder method attribute.
%output_builder_method%	An Optional parameter to provide the Name of a server method that will override GetScopeStructureOutputBase using the Aras.Server.Core.Configurator.ScopeStructureOutput template. The server method is used to extend responses using custom data. The AML targetScope item is accessible in the output_builder_method.

5.2.1.4 JSON Response Format

```
{
  "Item": {
    "@aras.type": "Scope",
    "@aras.id": "ScopeId",
    "name": "ScopeName",
    "Relationships": {
      "Item": [
        {
          "@aras.type": "Variable",
          "@aras.id": "VariableId_1",
          "id": "VariableId_1",
          "name": "VariableName_1",
          "Relationships": {
            "Item": [
              {
                "@aras.type": "NamedConstant",
                "@aras.id": "NamedConstantId_1",
                "id": "NamedConstantId_1",
                "name": "NamedConstantName_1"
              },
              {
                "@aras.type": "NamedConstant",
                "@aras.id": "NamedConstantId_2",
                "id": "NamedConstantId_2",
                "name": "NamedConstantName_2"
              }
            ]
          }
        }
      ]
    }
  },
  ...
  {
    "@aras.type": "Rule",
```

```

        "@aras.id": "RuleId_1",
        "id": "RuleId_1",
        "name": "RuleName_1",
        "definition": "expression"
    },
    ...
]
}
}
}
}

```

5.2.1.5 Examples

This section provides examples of AML and OData requests as well as AML and JSON responses for the same Scope Structure retrieved by the `cfg_GetScopeStructure` method.

- **AML Request**

```

<Item type="Method" action="cfg_GetScopeStructure">
  <targetScope>
    <Item type="Method" id="item_id_componentA" action="builder_method_name" />
  </targetScope>
</Item>

```

- **AML Response**

```

<Item type="Scope" id="item_id_componentA">
  <name>component A</name>
  <Relationships>
    <Item type="Variable" id="item_id_color">
      <id>item_id_color</id>
      <name>Color</name>
      <Relationships>
        <Item type="NamedConstant" id="item_id_red">
          <id>item_id_red</id>
          <name>Red</name>
        </Item>
        <Item type="NamedConstant" id="item_id_green">
          <id>item_id_green</id>
          <name>Green</name>
        </Item>
      </Relationships>
    </Item>
    <Item type="Variable" id="item_id_wheelsize">
      <id>item_id_wheelsize</id>
      <name>Wheel Size</name>
      <Relationships>
        <Item type="NamedConstant" id="item_id_15inch">
          <id>item_id_15inch</id>
          <name>15 inch</name>
        </Item>
        <Item type="NamedConstant" id="item_id_16inch">
          <id>item_id_16inch</id>
          <name>16 inch</name>
        </Item>
      </Relationships>
    </Item>
  </Relationships>
</Item>

```

```
<Item type="Rule" id="item_id_rule1">
  <id>item_id_rule1</id>
  <name>Rule Name 1</name>
  <definition>expression</definition>
</Item>
</Relationships>
</Item>
```

- **OData Request**

```
{
  "targetScope": {
    "Item": {
      "@aras.type": "Method",
      "@aras.action": "builder_method_name",
      "@aras.id": "item_id_componentA"
    }
  }
}
```

- **JSON Response**

```
{
  "Item": {
    "@aras.type": "Scope",
    "@aras.id": "item_id_componentA",
    "name": "component A",
    "Relationships": {
      "Item": [
        {
          "@aras.type": "Variable",
          "@aras.id": "item_id_color",
          "id": "item_id_color",
          "name": "Color",
          "Relationships": {
            "Item": [
              {
                "@aras.type": "NamedConstant",
                "@aras.id": "item_id_red",
                "id": "item_id_red",
                "name": "Red"
              },
              {
                "@aras.type": "NamedConstant",
                "@aras.id": "item_id_green",
                "id": "item_id_green",
                "name": "Green"
              }
            ]
          }
        }
      ]
    }
  },
}
```

```

"@aras.type": "Variable",
"@aras.id": "item_id_wheelsize",
"id": "item_id_wheelsize",
"name": "Wheel Size",
"Relationships": {
  "Item": [
    {
      "@aras.type": "NamedConstant",
      "@aras.id": "item_id_15inch",
      "id": "item_id_15inch",
      "name": "15 inch"
    },
    {
      "@aras.type": "NamedConstant",
      "@aras.id": "item_id_16inch",
      "id": "item_id_16inch",
      "name": "16 inch"
    }
  ]
},
{
  "@aras.type": "Rule",
  "@aras.id": "item_id_rule1",
  "id": "item_id_rule1",
  "name": "Rule Name 1",
  "definition": "expression"
}
]
}
}
}
}
}

```

5.2.2 cfg_GetValidCombinations

This method is designed to find a list of valid combinations.

Parameters

Required	Optional
targetScope	fetch, offset, select, responseFormat (default is JSON), condition For Missing Terms affect, refer to 5.3 Influence of Missing Terms on the API results .

5.2.2.1 AML Request Syntax

The AML request syntax is as follows:

```
<Item>
  type="Method"
  action="cfg_GetValidCombinations"
  select="%variableIds%"
  offset="%offset_integer%"
  fetch="%fetch_integer%"
  responseFormat="%XML|JSON%">
  <targetScope>
    <Item type="Method" action="%builder method name%" %builder method attributes%>
      %builder method properties%
    </Item>
  </targetScope>
  <condition>
    <![CDATA[
      <expression>...</expression>
    ]]>
  </condition>
</Item>
```

Where:

%variableIds%	Comma-separated variable ID list. Use the “select” attribute to request specified variables only. By default, the “select” attribute is empty. The <code>cfg_GetValidCombinations</code> response contains a full list of the variables that exist in the Scope.
%offset_integer%	Defines the number of combinations that can be skipped. Default value is 0.
%fetch_integer%	Defines the number of combinations that are returned, starting from the offset position. The Default value is empty, which returns all valid combinations. It is a best practice to use <code>fetch='1'</code> for requests that do not need a full list of combinations.
%XML JSON%	Specifies whether the response format is XML or JSON. The default is JSON.
<condition>...</condition>	Node that contains the expression to add to the Scope before finding combinations. The Default value is empty. For Missing Terms affect, refer to 5.3 Influence of Missing Terms on the API results
targetScope %builder method name% %builder method attributes% %builder method properties%	Refer to 4.3.2 Input Item Format targetScope is required. For Missing Terms affect, refer to 5.3 Influence of Missing Terms on the API results .

5.2.2.2 AML Response Format

5.2.2.2.1 Overview

This section describes the response format in terms of the object model.

The Response object has two properties: combinations-meta and combinations.

- Combinations-meta has two properties:
 - Variables – an array of variable objects. The Variable has an ID property.
 - Values – is an array of value objects. The Value has an ID property.
- Combinations is an array of combination objects. The Combination has a value (integer or an array of integers) property. The value of this property is used as an index or a set of indexes for the Values array.

Note: The BehaviorType of a Variable determines the type of its Combination value. For *exactly-one* BehaviorType, the Combination value is a single integer. For *at-least-one* BehaviorType, the Combination value is an array of integers. For *at-most-one* BehaviorType, the Combination value can be either an empty array or an array with a single integer.

Note: The *exactly-one* BehaviorType is the default for a Variable. The response formats and examples are provided for this behavior type. Examples with responses for the *at-least-one* and *at-most-one* BehaviorTypes can be found in "[Variable with BehaviorType equals at-least-one](#)" and "[Variable with BehaviorType equals at-most-one](#)".

5.2.2.2.2 Building list of VariableId=ValueId pairs

Each element in the combination array can be translated to VariableId=ValueId. The element Index in a combination array is an index of the appropriate elements in the Variables array. The Value of a combination element is an index of the appropriate elements in a Values array.

```
for (int index = 0; index < combination.Length; index++)
{
    string VariableId = Variables[index];
    string ValueId = Values[combination[index]];
}
```

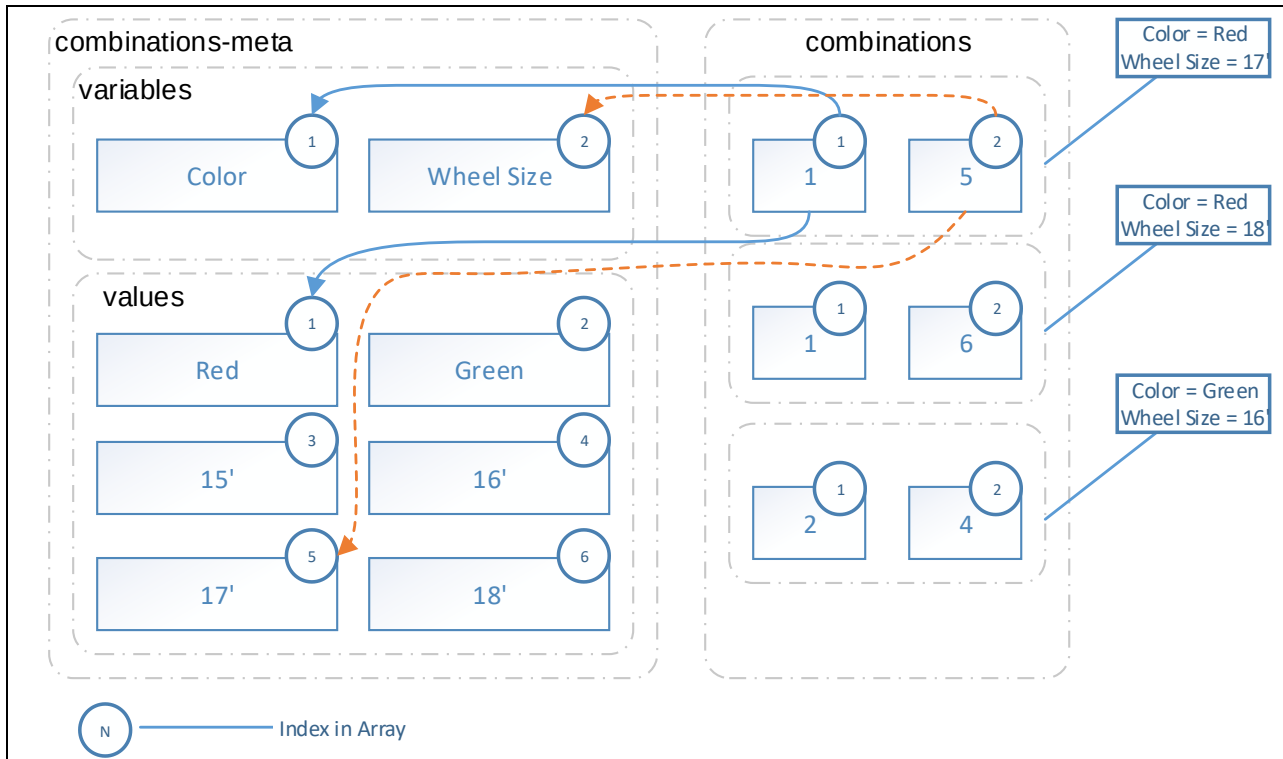


Figure 3.

5.2.2.2.3 XML Format

The following response in XML Format is a response object serialized to XML and wrapped to the AML Result:

```
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Body>
    <Result>
      <combinations-meta>
        <variables>
          <variable id="variable0Id" order="0" />
          <variable id="variable1Id" order="1" />
          ...
          <variable id="variableNId" order="N" />
        </variables>
        <values>
          <value type="valueType" order="0">value0Id</value>
          <value type="valueType" order="1">value1Id</value>
          ...
          <value type="valueType" order="M">valueMId</value>
        </values>
      </combinations-meta>
      <combinations>
        <combination>
          <value>valueIndex1</value>
          <value>valueIndex2</value>
          ...
          <value>valueIndexN</value>
        </combination>
      </combinations>
    </Result>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

```

        <combination>
            <value>valueIndex1</value>
            <value>valueIndex2</value>
            ...
            <value>valueIndexN</value>
        </combination>
        ...
        <combination>
            <value>valueIndex1</value>
            <value>valueIndex2</value>
            ...
            <value>valueIndexN</value>
        </combination>
    </combinations>
</Result>
</SOAP-ENV:Body>
</SOAP-ENV:Envelope>

```

- The <combinations-meta> node contains the <variables> and <values> child nodes.
- The <variables> node consists of child <variable> nodes, which are a list of variables contained in the current Scope. Each <variable> node contains the '@id' attribute, which defines the variable ID, and the '@order' attribute, which defines the order of the current variable within the <variables> node. Variable order starts at 0.
- The <values> node consists of a series of <value> child nodes, which list all of the values contained in the current Scope. Each <value> node has a '@type' attribute and an '@order' attribute. The '@type' attribute defines the value type (for example, 'Named Constant', 'Constant'). The '@order' attribute specifies the order of the current value within the <values> node. The Value order starts at 0. The Inner text of the <value> node is the ID of the Named Constant in the current Scope.
- The '<combinations>...</combinations>' node is a list of found combinations.
- The '<combination>...</combination>' node represents one combination.
- To determine a single combination, you should use <combination> and run through the <values>. For each <value> build a "Variable = Value" pair where the Variable and the Value can be found in '<Combinations-meta></combinations-meta>'.

Each <value>...</value> node within the '<combinations-meta><values></values></combinations-meta>' must be of type Named Constant.

If there are no valid combinations, the result node will be empty:

```

<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
    <SOAP-ENV:Body>
        <Result />
    </SOAP-ENV:Body>
</SOAP-ENV:Envelope>

```

5.2.2.2.4 JSON Format

The following response in JSON Format is a response object serialized to JSON and wrapped to the AML Result:

```
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Body>
    <Result>
      {
        "combinations-meta": {
          "variables": {
            "variable0Id": 0,
            "variable1Id": 1,
            ...
            "variableNId": N
          },
          "values": [{
            "type": "valueType",
            "id": "value0Id"
          },
          {
            "type": "valueType",
            "id": "value1Id"
          },
          ...
          {
            "type": "valueType",
            "id": "valueMId"
          }
        ],
        "combinations": [
          [valueIndex1,valueIndex2,...,valueIndexN],
          [valueIndex1,valueIndex2,...,valueIndexN],
          ...
          [valueIndex1,valueIndex2,...,valueIndexN]]
      }
    </Result>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

- The 'combinations-meta' object has the 'variables' and 'values' objects.
- The 'variables' object consists of name/value pairs, where name is the ID of the Variable in the current Scope and value defines the order of the current variable within the 'variables' object.
- The 'values' object is an array of objects that represent the Named Constant list for the current Scope.
- The 'combinations' object represents an array of valid combinations. Each element in the array represents a single combination.
- Each object within the '<values>' array is a Named Constant.

If there are no valid combinations, the result node will contain an empty JSON object:

```
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Body>
    <Result>{}</Result>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

5.2.2.3 Examples

- **Get all valid combinations with no limits**

The request is:

```
<Item type="Method" action="cfg_GetValidCombinations" responseFormat="XML">
  <targetScope>
    <Item type="Method" action="builder_method_name" id="business_item_id"/>
  </targetScope>
</Item>
```

The XML response:

```
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Body>
    <Result>
      <combinations-meta>
        <variables>
          <variable id="item_id_color" order="0" />
          <variable id="item_id_wheelsize" order="1" />
        </variables>
        <values>
          <value type="NamedConstant" order="0">item_id_red</value>
          <value type="NamedConstant" order="1">item_id_green</value>
          <value type="NamedConstant" order="2">item_id_15inch</value>
          <value type="NamedConstant" order="3">item_id_17inch</value>
        </values>
      </combinations-meta>
      <combinations>
        <combination>
          <value>0</value>
          <value>2</value>
        </combination>
        <combination>
          <value>0</value>
          <value>3</value>
        </combination>
        <combination>
          <value>1</value>
          <value>3</value>
        </combination>
        <combination>
          <value>1</value>
          <value>2</value>
        </combination>
      </combinations>
    </Result>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

The JSON response (@responseFormat="JSON" within the AML request):

```
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Body>
    <Result>
      {
```

```

    "combinations-meta": {
      "variables": {
        "item_id_color": 0,
        "item_id_wheelsize": 1
      },
      "values": [{
        "type": "NamedConstant",
        "id": "item_id_red"
      },
      {
        "type": "NamedConstant",
        "id": "item_id_green"
      },
      {
        "type": "NamedConstant",
        "id": "item_id_15inch"
      },
      {
        "type": "NamedConstant",
        "id": "item_id_17inch"
      }
    ]
  },
  "combinations": [[0,2],
    [0,3],
    [1,3],
    [1,2]]
}
</Result>
</SOAP-ENV:Body>
</SOAP-ENV:Envelope>

```

- **Using the @fetch attribute to get the first valid combination**

The request is:

```

<Item type="Method" action="cfg_GetValidCombinations" fetch="1" responseFormat="XML">
  <targetScope>
    <Item type="Method" action="builder_method_name" id="business_item_id"/>
  </targetScope>
</Item>

```

The XML response:

```

<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Body>
    <Result>
      <combinations-meta>
        <variables>
          <variable id="item_id_color" order="0" />
          <variable id="item_id_wheelsize" order="1" />
        </variables>
        <values>
          <value type="NamedConstant" order="0">item_id_red</value>
          <value type="NamedConstant" order="1">item_id_15inch</value>
        </values>
      </combinations-meta>
    </Result>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>

```

```

        <combinations>
          <combination>
            <value>0</value>
            <value>1</value>
          </combination>
        </combinations>
      </Result>
    </SOAP-ENV:Body>
  </SOAP-ENV:Envelope>

```

The JSON response (if @responseFormat="JSON" within the AML request):

```

<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Body>
    <Result>
      {
        "combinations-meta": {
          "variables": {
            "item_id_color": 0,
            "item_id_wheelsize": 1
          },
          "values": [{
            "type": "NamedConstant",
            "id": "item_id_red"
          },
          {
            "type": "NamedConstant",
            "id": "item_id_15inch"
          }
        ]
      },
      "combinations": [[0,1]]
    }
  </Result>
</SOAP-ENV:Body>
</SOAP-ENV:Envelope>

```

- **Using the @select attribute to get valid combinations for a single variable**

The request is:

```

<Item type="Method" action="cfg_GetValidCombinations" select="item_id_color"
responseFormat="XML">
  <targetScope>
    <Item type="Method" action="builder_method_name" id="business_item_id"/>
  </targetScope>
</Item>

```

The XML response:

```

<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Body>
    <Result>
      <combinations-meta>
        <variables>
          <variable id="item_id_color" order="0" />
        </variables>

```

```

        <values>
          <value type="NamedConstant" order="0">item_id_red</value>
          <value type="NamedConstant" order="1">item_id_green</value>
        </values>
      </combinations-meta>
    </combinations>
  </Result>
</SOAP-ENV:Body>
</SOAP-ENV:Envelope>

```

The JSON response (@responseFormat="JSON" within the AML request):

```

<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Body>
    <Result>
      {
        "combinations-meta": {
          "variables": {
            "item_id_color": 0
          },
          "values": [{
            "type": "NamedConstant",
            "id": "item_id_red"
          },
          {
            "type": "NamedConstant",
            "id": "item_id_green"
          }
        ]
      },
      "combinations": [[0],[1]]
    }
  </Result>
</SOAP-ENV:Body>
</SOAP-ENV:Envelope>

```

- **Missing Terms in condition**

The request contains "item_id_yellow" named constant, which is not included in the target Scope:

```

<Item type="Method" action="cfg_GetValidCombinations" select="item_id_color"
responseFormat="XML">
  <condition><![CDATA[
    <expression>
      <EQ>
        <variable id="item_id_color" />
        <named-constant id="item_id_yellow" />
      </EQ>
    </expression>
  ]]></condition>

```

```
<targetScope>
  <Item type="Method" action="builder_method_name" id="business_item_id"/>
</targetScope>
</Item>
```

The XML response:

```
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Body>
    <Result />
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

Refer to “5.3 Influence of Missing Terms on the API results”.

- **Variable with BehaviorType equals at-least-one**

The target Scope contains only one variable with BehaviorType=AtLeastOne with two named-constants.

The request is:

```
<Item type="Method" action="cfg_GetValidCombinations" responseFormat="XML">
  <targetScope>
    <Item type="Method" action="builder_method_name" id="business_item_id"/>
  </targetScope>
</Item>
```

The XML response:

```
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Body>
    <Result>
      <combinations-meta>
        <variables>
          <variable id="item_id_comfort" order="0"/>
        </variables>
        <values>
          <value type="NamedConstant" order="0">item_id_heatedseats</value>
          <value type="NamedConstant" order="1">item_id_cooledseats</value>
        </values>
      </combinations-meta>
      <combinations>
        <combination>
          <value>0</value>
        </combination>
        <combination>
          <value>1,0</value>
        </combination>
        <combination>
          <value>1</value>
        </combination>
      </combinations>
    </Result>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

The JSON response:

```
< SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Body>
    <Result>{
      "combinations-meta": {
        "variables": {
          "item_id_comfort": 0
        },
        "values": [
          {
            "type": "NamedConstant",
            "id": "item_id_heatedseats"
          },
          {
            "type": "NamedConstant",
            "id": "item_id_cooledseats"
          }
        ]
      },
      "combinations": [
        [[0]],
        [[1,0]],
        [[1]]
      ]
    }</Result>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

- **Variable with BehaviorType equals at-most-one**

The target Scope contains only one variable with BehaviorType=AtMostOne with two named-constants.

The request is:

```
<Item type="Method" action="cfg_GetValidCombinations" responseFormat="XML">
  <targetScope>
    <Item type="Method" action="builder_method_name" id="business_item_id"/>
  </targetScope>
</Item>
```

The XML response:

```
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Body>
    <Result>
      <combinations-meta>
        <variables>
          <variable id="item_id_comfort" order="0"/>
        </variables>
        <values>
          <value type="NamedConstant" order="0">item_id_heatedseats</value>
          <value type="NamedConstant" order="1">item_id_cooledseats</value>
        </values>
      </combinations-meta>
      <combinations>
        <combination>
          <value/>
        </combination>
      </combinations>
    </Result>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

```

        <combination>
          <value>0</value>
        </combination>
        <combination>
          <value>1</value>
        </combination>
      </combinations>
    </Result>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>

```

The JSON response:

```

<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Body>
    <Result>{
      "combinations-meta": {
        "variables": {
          "item_id_comfort": 0
        },
        "values": [
          {
            "type": "NamedConstant",
            "id": "item_id_heatedseats"
          },
          {
            "type": "NamedConstant",
            "id": "item_id_cooledseats"
          }
        ]
      },
      "combinations": [
        [[]],
        [[0]],
        [[1]]
      ]
    }</Result>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>

```

5.2.3 cfg_ValidateScope

The `cfg_ValidateScope` method checks to see if the built Scope has at least one valid combination. It also checks to see if each value can be assigned to a Variable for at least one valid combination.

5.2.3.1 AML Request Syntax

```

<Item type="Method" action="cfg_ValidateScope">
  <targetScope>
    <Item type="Method" action="%builder method name%" %builder method attributes%>
      %builder method properties%
    </Item>
  </targetScope>
</Item>

```

targetScope - % <i>builder method name</i> % - % <i>builder method attributes</i> % - % <i>builder method properties</i> %	Refer to 4.3.2 Input Item Format targetScope is required. For Missing Terms affect, refer to 5.3 Influence of Missing Terms on the API results
--	--

5.2.3.2 AML Response Format

The Response contains the Validation status and a description of the error:

1. Validation passed:

```
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Body>
    <Result>
      <Validation type="Validation" result="true"/>
    </Result>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

1. Validation failed – Scope has no valid combinations:

```
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Body>
    <Result>
      <Validation type="Validation" result="false">
        <Error>
          <Name>Validate Scope</Name>
          <Message>The problem does not have an optimal solution!</Message>
        </Error>
      </Validation>
    </Result>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

2. Validation failed – some values can't be assigned:

```
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Body>
    <Result>
      <Validation type="Validation" result="false">
        <Error>
          <Name>Scope Named Constants Availability Validate</Name>
          <Message>2 values are not available</Message>
          <details>
            <EQ>
              <variable name="Variable1" id="item_id_color" />
              <named-constant name="Value1" id="item_id_red" />
            </EQ>
            <EQ>
              <variable name="Variable1" id="item_id_wheelsize" />
              <named-constant name="Value2" id="item_id_16inch" />
            </EQ>
          </details>
        </Error>
      </Validation>
    </Result>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

5.2.4 cfg_GetIntersectingExpressions

The `cfg_GetIntersectingExpressions` method enables you to calculate the intersections between two or more expressions.

- The least complicated case is to check to see if the specified expressions are intersecting.
- The moderately complicated case is to find a pair of expressions from a list that are intersecting.
- The most complicated case is to find a combination of expressions from a list that are intersecting.

The first thing to do to resolve these tasks is to define a full list of expression combinations that need to be checked. After defining the expression combinations, validate each expression combination.

Note: Two or more expressions are intersecting if at least one valid combination exists that satisfies each expression.

5.2.4.1 AML Request Syntax

The AML request syntax is as follows:

```
<Item type="Method" action="cfg_GetIntersectingExpressions" responseFormat="%XML|JSON">
  <targetScope>
    <Item type="Method" action="%builder method name%" %builder method attributes%>
      %builder method properties%
    </Item>
  </targetScope>
  <cartesian-product>
    <set>
      <expression id="e1"><![CDATA[<expression>...</expression>]]></expression>
```

```

    <expression id="e2"><![CDATA[<expression>...</expression>]]></expression>
    ...
    <expression id="eN"><![CDATA[<expression>...</expression>]]></expression>
</set>
<set>
    <expression id="eE1"><![CDATA[<expression>...</expression>]]></expression>
    <expression id="eE2"><![CDATA[<expression>...</expression>]]></expression>
    ...
    <expression id="eEY"><![CDATA[<expression>...</expression>]]></expression>
</set>
...
<set>
    <expression id="eM1"><![CDATA[<expression>...</expression>]]></expression>
    <expression id="eM2"><![CDATA[<expression>...</expression>]]></expression>
    ...
    <expression id="eMZ"><![CDATA[<expression>...</expression>]]></expression>
</set>

```

You must include at least two sets within the 'cartesian-product' node

```

</cartesian-product>
</Item>

```

%XML JSON%	Specifies whether the response format is XML or JSON. The default is JSON.
targetScope %builder method name% %builder method attributes% %builder method properties%	Refer to 4.3.2 Input Item Format targetScope is required. For information about Missing Terms, refer to 5.3 Influence of Missing Terms on the API results
<cartesian-product>...</cartesian-product>	The node that contains sets of expressions that need to be verified for intersection within the provided Scope.
<set>..</set>	The container for expressions that is used to verify intersections with expressions contained within other <set>...</set> nodes. Since expressions within a single <set>...</set> node are not verified, you must provide at least two <set>...</set> nodes. For Missing Terms affect, refer to 5.3 Influence of Missing Terms on the API results

5.2.4.1.1 *Algorithm overview*

1. Take one expression from each <set>.
2. Concatenate the selected expressions in one expression using <AND>.
3. Validate the concatenated expression for the current Scope.
4. Repeat steps 1-3 for new expressions selection.

The Selected expressions are intersecting if there is at least one valid combination for Scope.

The following examples demonstrate how the algorithm prepares sets of expressions:

In this example, there are two sets of expressions, and each set contains one element:

A (a1), B (b1)

The algorithm creates the following set:

(a1, b1), (b1, a1)

There are two possible sets. They contain the same elements, but in a different order. The result looks like this:

(a1, b1)

In the following example, there are five sets and each of them contains one element:

A (a1), B (b1), C (c1), D (d1), E (e1)

The algorithm prepares the following sets:

(a1, b1), (a1, c1), (a1, d1), (a1, e1)
 (b1, a1), (b1, c1), (b1, d1), (b1, e1)
 (c1, a1), (c1, b1), (c1, d1), (c1, e1)
 (d1, a1), (d1, b1), (d1, c1), (c1, e1)
 (e1, a1), (e1, b1), (e1, c1), (e1, d1)

There are 20 possible sequences. Some of them contain the same elements, but they are ordered differently. The result looks like this:

(a1, b1), (a1, c1), (a1, d1), (a1, e1)
 (b1, c1), (b1, d1), (b1, e1)
 (c1, d1), (c1, e1)
 (d1, e1)

In the following example, there are three sets and each of them contains two elements:

A (a1, a2), B (b1, b2), C (c1, c2)

The algorithm prepares the following sets:

(a1, b1, c1), (a1, b1, c2), (a1, b2, c1), (a1, b2, c2)
 (a2, b1, c1), (a2, b1, c2), (a2, b2, c1), (a2, b2, c2)
 (b1, a1, c1), (b1, a1, c2), (b1, a2, c1), (b1, a2, c2)
 (b2, a1, c1), (b2, a1, c2), (b2, a2, c1), (b2, a2, c2)
 (c1, a1, b1), (c1, a1, b2), (c1, a2, b1), (c1, a2, b2)
 (c2, a1, b1), (c2, a1, b2), (c2, a2, b1), (c2, a2, b2)

There are 24 possible sequences and some of them contain the same elements, but they are ordered differently. The result looks like this:

(a1, b1, c1), (a1, b1, c2), (a1, b2, c1), (a1, b2, c2)
 (a2, b1, c1), (a2, b1, c2), (a2, b2, c1), (a2, b2, c2)

5.2.4.1.2 Cartesian Square AML Request Syntax

```
<Item type="Method" action="cfg_GetIntersectingExpressions" responseFormat="%XML|JSON%">
  <targetScope>
    <Item type="Method" action="%builder method name%" %builder method attributes%
      %builder method properties%
    </Item>
  </targetScope>
  <cartesian-square>
    <expression id="e1"><![CDATA[<expression>...</expression>]]></expression>
    <expression id="e2"><![CDATA[<expression>...</expression>]]></expression>
    ...
    <expression id="eN"><![CDATA[<expression>...</expression>]]></expression>
  </cartesian-square>
</Item>
```

Note: A minimum of two expressions is required within the 'cartesian-square' node.

%XML JSON%	Specifies whether the response format is XML or JSON. The default is JSON.
targetScope - %builder method name% - %builder method attributes% - %builder method properties%	Refer to "4.3.2 Input Item Format" targetScope is required For Missing Terms affect, refer to "5.3 Influence of Missing Terms on the API results"
<cartesian-square>...</cartesian-square>	Node that contains expressions.

Inside the API method, the Cartesian square is converted to a Cartesian product with the same two sets. The list of expressions from the Cartesian square goes to each of the two sets of the Cartesian product.

```
<cartesian-square>
  <expression id="e1"><![CDATA[<expression>...</expression>]]></expression>
  <expression id="e2"><![CDATA[<expression>...</expression>]]></expression>
  ...
  <expression id="eN"><![CDATA[<expression>...</expression>]]></expression>
</cartesian-square>
```

The previous expression is then transparently converted to:

```
<cartesian-product>
  <set>
    <expression id="e1"><![CDATA[<expression>...</expression>]]></expression>
    <expression id="e2"><![CDATA[<expression>...</expression>]]></expression>
    ...
    <expression id="eN"><![CDATA[<expression>...</expression>]]></expression>
  </set>
  <set>
    <expression id="e1"><![CDATA[<expression>...</expression>]]></expression>
    <expression id="e2"><![CDATA[<expression>...</expression>]]></expression>
    ...
    <expression id="eN"><![CDATA[<expression>...</expression>]]></expression>
  </set>
</cartesian-product>
```

The rest logic of the API method stays the same.

5.2.4.2 AML Response Format

The response is a list of corteges. A Cortege is a list of intersecting expressions. If there are no intersections, the result is empty.

5.2.4.2.1 XML format

```
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Body>
    <Result>
      <cortege>
        <expression id="e1" />
        <expression id="e2" />
        ...
        <expression id="eN" />
      </cortege>
      ...
      <cortege>
        <expression id="eM1" />
        <expression id="eM2" />
        ...
        <expression id="eMN" />
      </cortege>
    </Result>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

If there are no groups, the result node is empty:

```
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Body>
    <Result />
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

5.2.4.2.2 JSON format

```
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Body>
    <Result>
      [
        [
          {
            "id": "e1"
          },
          {
            "id": "e2"
          },
          ...
          {
            "id": "eN"
          }
        ],
        [
          {
            "id": "eE1"
          },
          {
            "id": "eE2"
          },
          ...
          {
            "id": "eEN"
          }
        ],
        [
          {
            "id": "eM1"
          },
          {
            "id": "eM2"
          },
          ...
          {
            "id": "eMN"
          }
        ]
      ]
    </Result>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

A
cortege

An expression

If there are no groups, the result node displays an empty JSON array:

```
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Body>
    <Result>[]</Result>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

5.2.4.3 Examples

- Calculating the intersection of two sets with a single expression in each one

```
<Item type="Method" action="cfg_GetIntersectingExpressions" responseFormat="XML">
  <targetScope>
    <Item type="Method" action="builder_method_name" id="business_item_id"/>
  </targetScope>
  <cartesian-product>
    <set>
      <expression id="e1">
        <![CDATA[
          <expression>
            <EQ>
              <variable id="item_id_color" />
              <named-constant id="item_id_red" />
            </EQ>
          </expression>
        ]]>
      </expression>
    </set>
    <set>
      <expression id="e2">
        <![CDATA[
          <expression>
            <EQ>
              <variable id="item_id_wheelsize" />
              <named-constant id="item_id_15inch" />
            </EQ>
          </expression>
        ]]>
      </expression>
    </set>
  </cartesian-product>
</Item>
```

The `cfg_GetIntersectingExpressions` method confirms that a Scope has valid combinations for ('e1' AND 'e2').

The XML response:

```
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Body>
    <Result>
      <cortege>
        <expression id="e1" />
        <expression id="e2" />
      </cortege>
    </Result>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

```

    </Result>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>

```

The JSON response (if @responseFormat="JSON" is within the AML request):

```

<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Body>
    <Result>
      [
        [
          {
            "id": "e1"
          },
          {
            "id": "e2"
          }
        ]
      ]
    </Result>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>

```

- Computing the intersection of two sets with two expressions in each one

```

<Item type="Method" action="cfg_GetIntersectingExpressions" responseFormat="XML">
  <targetScope>
    <Item type="Method" action="builder_method_name" id="business_item_id"/>
  </targetScope>
  <cartesian-product>
    <set>
      <expression id="e1">
        <![CDATA[
          <expression>
            <EQ>
              <variable id="item_id_color" />
              <named-constant id="item_id_red" />
            </EQ>
          </expression>
        ]]>
      </expression>
      <expression id="e2">
        <![CDATA[
          <expression>
            <EQ>
              <variable id="item_id_color" />
              <named-constant id="item_id_green" />
            </EQ>
          </expression>
        ]]>
      </expression>
    </set>
    <set>
      <expression id="e3">
        <![CDATA[
          <expression>

```

```

        <EQ>
          <variable id="item_id_wheelsize" />
          <named-constant id="item_id_15inch" />
        </EQ>
      </expression>
    ]]>
  </expression>
  <expression id="e4">
    <![CDATA[
      <expression>
        <EQ>
          <variable id="item_id_wheelsize" />
          <named-constant id="item_id_16inch" />
        </EQ>
      </expression>
    ]]>
  </expression>
</set>
</cartesian-product>
</Item>

```

Let us say the `cfg_GetIntersectingExpressions` method confirms that a Scope has the following available combinations:

1. 'e1' AND 'e3' is a valid combination
2. 'e2' AND 'e3' is a valid combination
3. 'e2' AND 'e4' is a valid combination

The XML response:

```

<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Body>
    <Result>
      <cortege>
        <expression id="e1" />
        <expression id="e3" />
      </cortege>
      <cortege>
        <expression id="e2" />
        <expression id="e3" />
      </cortege>
      <cortege>
        <expression id="e2" />
        <expression id="e4" />
      </cortege>
    </Result>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>

```

The JSON response (if `@responseFormat="JSON"` is within the AML request):

```

<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Body>
    <Result>

```

```
[
  [
    {
      "id": "e1"
    },
    {
      "id": "e3"
    }
  ],
  [
    {
      "id": "e2"
    },
    {
      "id": "e3"
    }
  ],
  [
    {
      "id": "e2"
    },
    {
      "id": "e4"
    }
  ]
]
</Result>
</SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

- Calculating the Cartesian square of three expressions

```
<Item type="Method" action="cfg_GetIntersectingExpressions" responseFormat="XML">
  <targetScope>
    <Item type="Method" action="builder_method_name" id="business_item_id"/>
  </targetScope>
  <cartesian-square>
    <expression id="e1">
      <![CDATA[
        <expression>
          <EQ>
            <variable id="item_id_color" />
            <named-constant id="item_id_red" />
          </EQ>
        </expression>
      ]]>
    </expression>
    <expression id="e2">
      <![CDATA[
        <expression>
          <EQ>
            <variable id="item_id_wheelsize" />
            <named-constant id="item_id_18inch" />
          </EQ>
        </expression>
      ]]>
    </expression>
  </cartesian-square>
</Item>
```

```

</expression>
<expression id="e3">
  <![CDATA[
    <expression>
      <EQ>
        <variable id="item_id_bodytype" />
        <named-constant id="item_id_sport" />
      </EQ>
    </expression>
  ]]>
</expression>
</cartesian-square>
</Item>

```

Let us say the `cfg_GetIntersectingExpressions` method confirms that a Scope has the following available combinations:

1. 'e1' AND 'e2' is a valid combination
2. 'e1' AND 'e3' is a valid combination
3. 'e2' AND 'e3' is a valid combination

The XML response:

```

<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Body>
    <Result>
      <cortege>
        <expression id="e1" />
        <expression id="e2" />
      </cortege>
      <cortege>
        <expression id="e1" />
        <expression id="e3" />
      </cortege>
      <cortege>
        <expression id="e2" />
        <expression id="e3" />
      </cortege>
    </Result>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>

```

The JSON response (if `@responseFormat="JSON"` within the AML request):

```

<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Body>
    <Result>
      [
        [
          {
            "id": "e1"
          },
          {
            "id": "e2"
          }
        ]
      ]
    </Result>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>

```

```
    }  
  ],  
  [  
    {  
      "id": "e1"  
    },  
    {  
      "id": "e3"  
    }  
  ],  
  [  
    {  
      "id": "e2"  
    },  
    {  
      "id": "e3"  
    }  
  ]  
]  
</Result>  
</SOAP-ENV:Body>  
</SOAP-ENV:Envelope>
```

5.2.5 cfg_GetConflicts

The API method described in this section will help you to find the reason why the Scope does not have a solution. Use the `cfg_GetConflicts` internal server method to find and describe all contradictions between existing rules, variable constraints, and custom conditions.

The method returns a list of found conflicts. Each conflict contains a list of conflict sources (rules, variables, missing items, conditions).

5.2.5.1 AML Request Syntax

The AML request syntax is as follows:

```
<Item type="Method" action="cfg_GetConflicts" fetch="%fetch_integer%">  
  <targetScope>  
    <Item type="Method" action="%builder method name%" %builder method attributes%>  
      %builder method properties%  
    </Item>  
  </targetScope>  
  <condition>...</condition>  
</Item>
```

%fetch_integer%	Defines the number of conflicts that are returned. The Default value is empty, which returns all conflicts. It is suggested to use a low number for the 'fetch' based on how many conflicts need to be returned based on the business use cases. If a high or no value is specified for 'fetch' and the scope is complex, the execution time to return these conflicts will be high.
------------------------	---

targetScope - % <i>builder method name</i> % - % <i>builder method attributes</i> % - % <i>builder method properties</i> %	Refer to 4.3.2 Input Item Format targetScope is required. For Missing Terms affect, refer to 5.3 Influence of Missing Terms on the API results .
<condition>...</condition>	A Node that contains the expression to add to the Scope before finding combinations. The expression is included in the list of conflict sources. The Default value is empty. For Missing Terms affect, refer to 5.3 Influence of Missing Terms on the API results .

5.2.5.2 AML Response Format

```

<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Body>
    <Result>
      <Conflict>
        <member source="condition" propositionalForm="...">
          <![CDATA[<expression>...</expression>]]>
        </member>
        <member source="variable" id="..." propositionalForm="...">
          <![CDATA[<expression>...</expression>]]>
        </member>
        <member source="rule" id="..." propositionalForm="...">
          <![CDATA[<expression>...</expression>]]>
        </member>
        <member source="missinginscope" propositionalForm="...">
          <![CDATA[<expression>...</expression>]]>
        </member>
      </Conflict>
      <Conflict>
        ...
      </Conflict>
      ...
    </Result>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>

```

The <Result> node contains a <Conflict> child node for each found conflict. The <Conflict> node consists of <member> child nodes that represent a certain expression that causes conflict.

The <member> node contains the following:

- source attribute stores the member type: rule, variable, condition or missinginscope.
- propositionalForm attribute holds the text representation of the member's inner expression.
- id attribute stores the corresponding Rule or Variable ID if the "source" attribute equals to "rule" or "variable".
- inner text, which contains the serialized expression of this conflict member.

Conflict source types are follows:

- rule corresponds to a Scope Rule.
- variable corresponds to a Scope Variable.
- condition corresponds to the expression specified as input.
- missinginscope corresponds to an expression containing a single equivalence. An equivalence is “missing” if one or both equivalence parts are not in Scope. The Configurator Services API considers such an equivalence to be False.

5.2.5.3 Examples

Scope contains the following variables:

- Color: Red, Green, Blue, Black, White
- Style: Wagon, Business, Sport
- Wheel Size: 14", 15", 16", 17"
- Wheel Type: Steel, Aluminum

Scope contains the following rules:

- “Red” is the only color available for “Sport” Style.
- “Business” is only available in the colors “Black” or “White”.
- “Wagon” Style is only available for “14” and “15” Wheel Size.
- “Sport” Style is only available for “16” and “17” Wheel Size.
- “14” Wheel Size is only available for the “Steel” Wheel Type.
- “17” Wheel Size is only available for the “Aluminum” Wheel Type.

This scope has the following valid combinations:

- Color=Green, Style=Wagon, WheelSize=14", WheelType=Steel.
- Color=White, Style=Business, WheelSize=15", WheelType=Steel.
- Color=Red, Style=Sport, WheelSize=17", WheelType=Aluminum.
- Valid Scope

Request:

```
<Item type="Method" action="cfg_GetConflicts">
  <targetScope>...</targetScope>
</Item>
```

Response:

Because the scope is solvable, the result is empty.

```
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Body>
    <Result />
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

- Valid Scope – check with condition

Request:

```
<Item type="Method" action="cfg_GetConflicts">
  <targetScope>...</targetScope>
  <condition><![CDATA[
    <expression>
      <EQ>
        <variable id="IDOFCOLOR" />
        <named-constant id="IDOFGREEN" />
      </EQ>
      <EQ>
        <variable id="IDOFSTYLE" />
        <named-constant id="IDOFWAGON" />
      </EQ>
      <EQ>
        <variable id="IDOFWHEELSIZE" />
        <named-constant id="IDOF14INCHES" />
      </EQ>
      <EQ>
        <variable id="IDOFWHEELTYPE" />
        <named-constant id="IDOFSTEEL" />
      </EQ>
    </expression>
  ]]></condition>
</Item>
```

Response:

Since the scope is solvable, the result is empty.

```
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Body>
    <Result />
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

- Simple single conflict

Request:

```
<Item type="Method" action="cfg_GetConflicts">
  <targetScope>...</targetScope>
  <condition><![CDATA[
    <expression>
      <EQ>
        <variable id="IDOFCOLOR" />
        <named-constant id="IDOFRED" />
      </EQ>
      <EQ>
        <variable id="IDOFSTYLE" />
        <named-constant id="IDOFWAGON" />
      </EQ>
      <EQ>
        <variable id="IDOFWHEELSIZE" />
        <named-constant id="IDOF14INCHES" />
      </EQ>
      <EQ>
        <variable id="IDOFWHEELTYPE" />
        <named-constant id="IDOFSTEEL" />
      </EQ>
    </expression>
  ]]></condition>
</Item>
```

```

        </EQ>
      </expression>
    ]]></condition>
  </Item>

```

Response:

The following condition contains the conflicting variants “Red” and “Wagon” and the rule, which makes their simultaneous usage impossible:

```

<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Body>
    <Result>
      <Conflict>
        <member source="condition" propositionalForm="..."><![CDATA[
          <expression>
            <EQ>
              <variable id="IDOFCOLOR" />
              <named-constant id="IDOFRED" />
            </EQ>
            <EQ>
              <variable id="IDOFSTYLE" />
              <named-constant id="IDOFWAGON" />
            </EQ>
          </expression>
        ]]></member>
        <member source="rule" id="1234" propositionalForm="IF Color=Red THEN
Style=Sport">...</member>
      </Conflict>
    </Result>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>

```

- Just one value can be set to Variable

Request:

```

<Item type="Method" action="cfg_GetConflicts">
  <targetScope>...</targetScope>
  <condition><![CDATA[
    <expression>
      <EQ>
        <variable id="IDOFCOLOR" />
        <named-constant id="IDOFRED" />
      </EQ>
      <EQ>
        <variable id="IDOFCOLOR" />
        <named-constant id="IDOFGREEN" />
      </EQ>
    </expression>
  ]]></condition>
</Item>

```

Response:

These two values contradict each other because both belong to the "Color" Variable. The Variable cannot contain two values simultaneously.

```
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Body>
    <Result>
      <Conflict>
        <member source="condition" propositionalForm="Color=Red AND
Color=Green"><![CDATA[
          <expression>
            <EQ>
              <variable id="IDOFCOLOR" />
              <named-constant id="IDOFRED" />
            </EQ>
            <EQ>
              <variable id="IDOFCOLOR" />
              <named-constant id="IDOFGREEN" />
            </EQ>
          </expression>
        ]]></member>
        <member source="variable" id="IDOFCOLOR" propositionalForm="EXACTLY-ONE
(Color=Red | Color=Green | Color=Blue | Color=Black | Color = White)">...</member>
      </Conflict>
    </Result>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

- Variable should have value

Request:

```
<Item type="Method" action="cfg_GetConflicts">
  <targetScope>...</targetScope>
  <condition><![CDATA[
    <expression>
      <NOT>
        <EQ>
          <variable id="IDOFSTYLE" />
          <named-constant id="IDOFWAGON" />
        </EQ>
      </NOT>
      <NOT>
        <EQ>
          <variable id="IDOFSTYLE" />
          <named-constant id="IDOBUSINESS" />
        </EQ>
      </NOT>
      <NOT>
        <EQ>
          <variable id="IDOFSTYLE" />
          <named-constant id="IDOFSPORT" />
        </EQ>
      </NOT>
    </expression>
  ]]></condition>
```

```
</Item>
```

Response:

The following three values are the full list of possible values for the Style Variable. The condition says that the Style Variable cannot be any of these values. The Variable cannot be empty.

```
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Body>
    <Result>
      <Conflict>
        <member source="condition" propositionalForm="NOT (Style=Wagon) AND NOT
(Style=Business) AND NOT (Style=Sport)" ><![CDATA[
          <expression>
            <NOT>
              <EQ>
                <variable id="IDOFSTYLE" />
                <named-constant id="IDOFWAGON" />
              </EQ>
            </NOT>
            <NOT>
              <EQ>
                <variable id="IDOFSTYLE" />
                <named-constant id="IDOBUSINESS" />
              </EQ>
            </NOT>
            <NOT>
              <EQ>
                <variable id="IDOFSTYLE" />
                <named-constant id="IDOFSPORT" />
              </EQ>
            </NOT>
          </expression>]]>
        </member>
        <member source="variable" id="IDOFSTYLE" propositionalForm=" EXACTLY-ONE
(Style=Wagon | Style=Business | Style=Sport)">...</member>
      </Conflict>
    </Result>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

- Multiple conflicts

Request:

```
<Item type="Method" action="cfg_GetConflicts">
  <targetScope>...</targetScope>
  <condition><![CDATA[
    <expression>
      <EQ>
        <variable id="IDOFCOLOR" />
        <named-constant id="IDOFRED" />
      </EQ>
      <EQ>
        <variable id="IDOFSTYLE" />
        <named-constant id="IDOFWAGON" />
      </EQ>
    </expression>
  </condition>
```

```

    </EQ>
    <EQ>
      <variable id="IDOFWHEELSIZE" />
      <named-constant id="IDOF17INCHES" />
    </EQ>
    <EQ>
      <variable id="IDOFWHEELTYPE" />
      <named-constant id="IDOFSTEEL" />
    </EQ>
  </expression>
]]></condition>
</Item>

```

Response:

This combination contains multiple contradictions. Each of them transforms into a <Conflict>-node.

```

<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Body>
    <Result>
      <Conflict>
        <member source="condition" propositionalForm="Style=Wagon AND
Color=Red"><![CDATA[
          <expression>
            <EQ>
              <variable id="IDOFCOLOR" />
              <named-constant id="IDOFRED" />
            </EQ>
            <EQ>
              <variable id="IDOFSTYLE" />
              <named-constant id="IDOFWAGON" />
            </EQ>
          </expression>
        ]]></member>
        <member source="rule" id="1234" propositionalForm="IF Color=Red THEN
Style=Sport">...</member>
      </Conflict>
      <Conflict>
        <member source="condition" propositionalForm='Style=Wagon AND [Wheel
Size]=[17"]'><![CDATA[
          <expression>
            <EQ>
              <variable id="IDOFSTYLE" />
              <named-constant id="IDOFWAGON" />
            </EQ>
            <EQ>
              <variable id="IDOFWHEELSIZE" />
              <named-constant id="IDOF17INCHES" />
            </EQ>
          </expression>
        ]]></member>
        <member source="rule" id="5678" propositionalForm='IF Style=Wagon THEN
([Wheel Size]=[14"] OR [Wheel Size]=[15"])'>...</member>
      </Conflict>
    </Conflict>
  </Result>

```

```

        <member source="condition" propositionalForm='[Wheel Type]=Steel AND [Wheel
Size]=[17"]'><![CDATA[
            <expression>
                <EQ>
                    <variable id="IDOFWHEELSIZE" />
                    <named-constant id="IDOF17INCHES" />
                </EQ>
                <EQ>
                    <variable id="IDOFWHEELTYPE" />
                    <named-constant id="IDOFSTEEL" />
                </EQ>
            </expression>
        ]]></member>
        <member source="rule" id="0011" propositionalForm='IF [Wheel Size]=[17"]
THEN [Wheel Type]= Aluminum'>...</member>
    </Conflict>
</Result>
</SOAP-ENV:Body>
</SOAP-ENV:Envelope>

```

- Using the @fetch attribute to get one conflict

Request:

```

<Item type="Method" action="cfg_GetConflicts" fetch="1">
    <targetScope>...</targetScope>
    <condition><![CDATA[
        <expression>
            <EQ>
                <variable id="IDOFCOLOR" />
                <named-constant id="IDOFRED" />
            </EQ>
            <EQ>
                <variable id="IDOFSTYLE" />
                <named-constant id="IDOFWAGON" />
            </EQ>
            <EQ>
                <variable id="IDOFWHEELSIZE" />
                <named-constant id="IDOF17INCHES" />
            </EQ>
            <EQ>
                <variable id="IDOFWHEELTYPE" />
                <named-constant id="IDOFSTEEL" />
            </EQ>
        </expression>
    ]]></condition>
</Item>

```

Response:

```

<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
    <SOAP-ENV:Body>
        <Result>
            <Conflict>
                <member source="condition" propositionalForm="Style=Wagon AND
Color=Red"><![CDATA[

```

```

        <expression>
          <EQ>
            <variable id="IDOFCOLOR" />
            <named-constant id="IDOFRED" />
          </EQ>
          <EQ>
            <variable id="IDOFSTYLE" />
            <named-constant id="IDOFWAGON" />
          </EQ>
        </expression>
      ]]></member>
      <member source="rule" id="1234" propositionalForm="IF Color=Red THEN
Style=Sport">...</member>
    </Conflict>
  </Result>
</SOAP-ENV:Body>
</SOAP-ENV:Envelope>

```

- Unobvious conflicts

Variables: "Size" { "Big", "Small" } and "Color" { "Black", "White" }

Rules:

"IF (Size=Big) OR (Color=Black) THEN (Size=Big) AND (Color=Black)",

"IF (Size=Small) OR (Color=White) THEN (Size=Small) AND (Color=White)".

This will have two valid combinations: "Big", "Black" and "Small", "White".

When we try the condition (Color=White) AND (Color=Black), we will get two conflicts instead of one:

Request:

```

<Item type="Method" action="cfg_GetConflicts">
  <targetScope>...</targetScope>
  <condition><![CDATA[
    <expression>
      <EQ>
        <variable id="IDOFSIZE" />
        <named-constant id="IDOFBIG" />
      </EQ>
      <EQ>
        <variable id="IDOFSIZE" />
        <named-constant id="IDOFSMALL" />
      </EQ>
    </expression>
  ]]></condition>
</Item>

```

Response:

```

<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Body>
    <Result>
      <Conflict><!-- This conflict is obvious and expected. -->
        <member source="condition" propositionalForm="Size=Big AND
Size=Small"><![CDATA[
          <expression>
            <EQ>

```

```

        <variable id="IDOF SIZE" />
        <named-constant id="IDOFBIG" />
    </EQ>
    <EQ>
        <variable id="IDOF SIZE" />
        <named-constant id="IDOF SMALL" />
    </EQ>
</expression>
]]></member>
<member source="variable" id="IDOF SIZE" propositionalForm="EXACTLY-ONE
(Size=Big | Size = Small)">...</member>
</Conflict>
<Conflict>
    <member source="condition" propositionalForm="Size=Big and
Size=Small"><![CDATA[
        <expression>
        <EQ>
            <variable id="IDOF SIZE" />
            <named-constant id="IDOFBIG" />
        </EQ>
        <EQ>
            <variable id="IDOF SIZE" />
            <named-constant id="IDOF SMALL" />
        </EQ>
        </expression>
    ]]></member>
    <member source="variable" id="IDOCOLOR" propositionalForm="EXACTLY-ONE
(Color=Black | Color=White)">...</member>
    <member source="rule" id="XXYYZZ" propositionalForm="IF (Size=Big) OR
(Color=Black) THEN (Size=Big) AND (Color=Black)">...</member>
    <member source="rule" id="UUVVWW" propositionalForm="IF (Size=Small) OR
(Color=White) THEN (Size=Small) AND (Color=White)">...</member>
</Conflict>
</Result>
</SOAP-ENV:Body>
</SOAP-ENV:Envelope>

```

The explanation of the last conflict is as follows:

- Value “Big” and Rule “IF (Size=Big) OR (Color=Black) THEN (Size=Big) AND (Color=Black)” together means that the “Black” value must be selected.
- Value “Small” and Rule “IF (Size=Small) OR (Color=White) THEN (Size=Small) AND (Color=White)” together means that “White” value must be selected.
- The colors Black and White cannot be chosen at the same time “EXACTLY-ONE (Color=Black | Color=White)”. Only one value can be set for the variable.
- Using a Boolean expression as a condition

It is possible to use a complex Boolean expression as an input parameter:

Request:

```

<Item type="Method" action="cfg_GetConflicts">
<targetScope>...</targetScope>
    <condition><![CDATA[<expression>

```

```

<OR>
  <AND>
    <EQ>
      <variable id="IDOFSIZE" />
      <named-constant id="IDOFBIG" />
    </EQ>
    <EQ>
      <variable id="IDOFCOLOR" />
      <named-constant id="IDOFWHITE" />
    </EQ>
  </AND>
  <AND>
    <EQ>
      <variable id="IDOFSIZE" />
      <named-constant id="IDOFSMALL" />
    </EQ>
    <EQ>
      <variable id="IDOFCOLOR" />
      <named-constant id="IDOFBLACK" />
    </EQ>
  </AND>
</OR>
</expression>]]></condition>
</Item>

```

This condition describes the disjunction of two invalid combinations.

Response:

```

<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Body>
    <Result>
      <Conflict>
        <member source="condition" propositionalForm="(Size=Big AND Color=White) OR
(Size=Small AND Color=Black)"><![CDATA[<expression>
  <OR>
    <AND>
      <EQ>
        <variable id="IDOFSIZE" />
        <named-constant id="IDOFBIG" />
      </EQ>
      <EQ>
        <variable id="IDOFCOLOR" />
        <named-constant id="IDOFWHITE" />
      </EQ>
    </AND>
    <AND>
      <EQ>
        <variable id="IDOFSIZE" />
        <named-constant id="IDOFSMALL" />
      </EQ>
      <EQ>
        <variable id="IDOFCOLOR" />
        <named-constant id="IDOFBLACK" />
      </EQ>
    </AND>
  </OR>
</expression>]]></condition>
</member>
      </Conflict>
    </Result>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>

```

```

        </EQ>
    </AND>
</OR>
</expression>]]></member>
    <member source="variable" id="IDOF SIZE" propositionalForm="EXACTLY-ONE
(Size=Big | Size = Small)">...</member>
    <member source="variable" id="IDOF COLOR" propositionalForm="EXACTLY-ONE
(Color=Black | Color = White)">...</member>
    <member source="rule" id="XXYYZZ" propositionalForm="IF (Size=Big) OR
(Color=Black) THEN (Size=Big) AND (Color=Black)">...</member>
</Conflict>
<Conflict>
    <member source="condition" propositionalForm="(Size=Big AND Color=White) OR
(Size=Small AND Color=Black)"><![CDATA[<expression>
    <OR>
        <AND>
            <EQ>
                <variable id="IDOF SIZE" />
                <named-constant id="IDOFBIG" />
            </EQ>
            <EQ>
                <variable id="IDOF COLOR" />
                <named-constant id="IDOFWHITE" />
            </EQ>
        </AND>
        <AND>
            <EQ>
                <variable id="IDOF SIZE" />
                <named-constant id="IDOF SMALL" />
            </EQ>
            <EQ>
                <variable id="IDOF COLOR" />
                <named-constant id="IDOFBLACK" />
            </EQ>
        </AND>
    </OR>
    </expression>]]></member>
    <member source="variable" id="IDOF SIZE" propositionalForm="EXACTLY-ONE
(Size=Big | Size = Small)">...</member>
    <member source="variable" id="IDOF COLOR" propositionalForm="EXACTLY-ONE
(Color=Black | Color = White)">...</member>
    <member source="rule" id="UUVVWW" propositionalForm="IF (Size=Small) OR
(Color=White) THEN (Size=Small) AND (Color=White)">...</member>
</Conflict>
<Conflict>
    <member source="condition" propositionalForm="(Size=Big AND Color=White) OR
(Size=Small AND Color=Black)"><![CDATA[<expression>
    <OR>
        <AND>
            <EQ>
                <variable id="IDOF SIZE" />
                <named-constant id="IDOFBIG" />

```

```

    </EQ>
    <EQ>
      <variable id="IDOFCOLOR" />
      <named-constant id="IDOFWHITE" />
    </EQ>
  </AND>
  <AND>
    <EQ>
      <variable id="IDOFSIZE" />
      <named-constant id="IDOFSMALL" />
    </EQ>
    <EQ>
      <variable id="IDOFCOLOR" />
      <named-constant id="IDOFBLACK" />
    </EQ>
  </AND>
</OR>
</expression>]]></member>
  <member source="variable" id="IDOFSIZE" propositionalForm="EXACTLY-ONE
(Size=Big | Size = Small)">...</member>
  <member source="rule" id="XXYYZZ" propositionalForm="IF (Size=Big) OR
(Color=Black) THEN (Size=Big) AND (Color=Black)">...</member>
  <member source="rule" id="UUVVWw" propositionalForm="IF (Size=Small) OR
(Color=White) THEN (Size=Small) AND (Color=White)">...</member>
</Conflict>
<Conflict>
  <member source="condition" propositionalForm="(Size=Big AND Color=White) OR
(Size=Small AND Color=Black)"><![CDATA[<expression>
  <OR>
    <AND>
      <EQ>
        <variable id="IDOFSIZE" />
        <named-constant id="IDOFBIG" />
      </EQ>
      <EQ>
        <variable id="IDOFCOLOR" />
        <named-constant id="IDOFWHITE" />
      </EQ>
    </AND>
    <AND>
      <EQ>
        <variable id="IDOFSIZE" />
        <named-constant id="IDOFSMALL" />
      </EQ>
      <EQ>
        <variable id="IDOFCOLOR" />
        <named-constant id="IDOFBLACK" />
      </EQ>
    </AND>
  </OR>
  </expression>]]></member>
  <member source="variable" id="IDOFCOLOR" propositionalForm="EXACTLY-ONE

```

```
(Color=Black | Color = White)">...</member>
  <member source="rule" id="XXYYZZ" propositionalForm="IF (Size=Big) OR
(Color=Black) THEN (Size=Big) AND (Color=Black)">...</member>
  <member source="rule" id="UUVVWw" propositionalForm="IF (Size=Small) OR
(Color=White) THEN (Size=Small) AND (Color=White)">...</member>
</Conflict>
</Result>
```

- Missing terms

The request contains an expression that specifies a color, which is out of scope:

```
<Item type="Method" action="cfg_GetConflicts">
  <targetScope>...</targetScope>
  <condition><![CDATA[<expression>
    <EQ>
      <variable id="IDOFCOLOR" />
      <named-constant id="YELLOW" />
    </EQ>
  </expression>]]></condition>
</Item>
```

This term causes the conflict, the corresponding `<member>` nodes to reflect this fact (refer to [5.3 Influence of Missing Terms on the API results](#)):

```
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Body>
    <Result>
      <Conflict>
        <member source="condition"
propositionalForm="Color=Yellow"><![CDATA[<expression>
  <EQ>
    <variable id="IDOFCOLOR" />
    <named-constant id="YELLOW" />
  </EQ>
</expression>]]>
        </member>
        <member source="missinginscope"
propositionalForm="NOT(Color=Yellow)">...</member>
      </Conflict>
    </Result>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

- Scope with no valid combinations

Scope with:

- Variable "Color" { "Black", "White" }.
- Variable "Size" { "Small" }.
- Rule "IF (Color=Black) OR (Color=White) THEN NOT (Size=Small)".

The Scope is invalid because the Variable cannot be assigned.

Request:

```
<Item type="Method" action="cfg_GetConflicts">
  <targetScope>...</targetScope>
  <condition />
</Item>
```

Response:

```
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Body>
    <Result>
      <Conflict>
        <member source="variable" id="IDOFCOLOR" propositionalForm="EXACTLY-ONE
(Color=Black | Color = White)">...</member>
        <member source="rule" id="UUUVWW" propositionalForm="IF (Color=Black) OR
(Color=White) THEN NOT (Size=Small)">...</member>
      </Conflict>
    </Result>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

Input condition expressions are not necessary to cause conflicts.

5.2.6 cfg_GetExpressionTruthTable

The `cfg_GetExpressionTruthTable` method is very helpful for debugging purposes. Analyzing and resolving the process can reveal the direct contradictions between different parts of the scope, which may not be so obvious.

- The method builds a truth table for the specified expression.
- The method works without the Scope being specified.
- The method auto generates Scope inside the method implementation.
- For each equivalence from the expression the new Variable is added to the auto generated Scope.
- All Variables in the auto generated Scope have two possible namedConstants: 0, 1.
- The Expression specified in the request translates to new Variables.

Example: for the expression (if Color=White then Color=Black) and (if Color=Black then Color=White)

AutoGeneratedScope

[Color=White]

1

0

[Color=Black]

1

0

Translated expression:

(IF [Color=White] = 1 THEN [Color=Black] = 1) AND (IF [Color=Black] = 1 THEN [Color=White] = 1)

The `cfg_GetExpressionTruthTable` method finds and returns all valid combinations for the new AutoGeneratedScope using the translated expression as a condition.

5.2.6.1 AML Request Syntax

```
<Item type="Method" action="cfg_GetExpressionTruthTable" responseFormat="%XML|JSON%">
  <condition>
    <![CDATA[
      <expression>...</expression>
    ]]>
  </condition>
</Item>
```

%XML JSON%	Specifies whether the response format is XML or JSON. The default is JSON.
<condition>...</condition>	A Node that contains the expression. The Default value is empty.

5.2.6.2 AML Response Format

```
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Body>
    <Result>
      <truth-table-meta>
        <terms>
          <term order="0"><![CDATA[<expression>...</expression>]]></term>
          <term order="1">...</term>
          ...
        </terms>
      </truth-table-meta>
      <truth-table>
        <combination>
          <value>1</value>
          <value>0</value>
          ...
        </combination>
        ...
      </truth-table>
    </Result>
  </SOAP-ENV:Body>
```

```
</SOAP-ENV:Envelope>
```

The method builds a response in the same way as `cfg_GetValidCombinations`.

The `<truth-table-meta>` consists of `<terms>`, where the `<term>` is an expression with the equivalence inside. The term itself is a Boolean variable with values of either 0 or 1. The `<truth-table>` consists of combinations, where each `<combination>` represents the values of terms that lead the initial condition to be true.

The JSON response (@responseFormat="JSON" within the AML request):

```
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Body>
    <Result>
      {
        "truth-table-meta": {
          terms: [
            { expression: "<eq>...</eq>" },
            ...
            { expression: "<eq>...</eq>" },
          ]
        },
        "truth-table": [
          [.., ..., ..],
          ...
          [.., ..., ..]
        ]
      }
    </Result>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

5.2.6.3 Examples

- Build truth table

Build a truth table for EXACTLY-ONE(TIRES_TYPE=[SC 3], TIRES_TYPE=[PC 5], TIRES_TYPE=[SC 5]).

There are three rows in the truth table:

TIRES_TYPE=[SC 3]	TIRES_TYPE=[PC 5],	TIRES_TYPE=[SC 5]
1	0	0
0	1	0
0	0	1

Request:

```
<Item type="Method" action="cfg_GetExpressionTruthTable">
  <condition><![CDATA[<expression>
    <exactly-one>
```

```

    <eq>
      <variable id="TIRES_TYPE" />
      <named-constant id="SC 3" />
    </eq>
    <eq>
      <variable id="TIRES_TYPE" />
      <named-constant id="PC 5" />
    </eq>
    <eq>
      <variable id="TIRES_TYPE" />
      <named-constant id="SC 5" />
    </eq>
  </exactly-one>
</expression>]]>
</condition>
</Item>

```

The XML response:

```

<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Body>
    <Result>
      <truth-table-meta>
        <terms>
          <term order="0">
            <![CDATA[<expression>
              <eq><variable id="TIRES_TYPE" /><named-constant id="SC 3" /></eq>
            </expression>]]>
          </term><term order="1">
            <![CDATA[<expression>
              <eq><variable id="TIRES_TYPE" /><named-constant id="PC 5" /></eq>
            </expression>]]>
          </term><term order="2">
            <![CDATA[<expression>
              <eq><variable id="TIRES_TYPE" /><named-constant id="SC 5" /></eq>
            </expression>]]>
          </term>
        </terms>
      </truth-table-meta>
      <truth-table>
        <combination>
          <value>1</value>
          <value>0</value>
          <value>0</value>
        </combination>
        <combination>
          <value>0</value>
          <value>1</value>
          <value>0</value>
        </combination>
        <combination>
          <value>0</value>
          <value>0</value>
          <value>1</value>
        </combination>
      </truth-table>
    </Result>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>

```

```

    </Result>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>

```

The JSON response (@responseFormat="JSON" within the AML request):

```

<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Body>
    <Result>
      {
        "truth-table-meta": {
          terms: [
            { expression: "<eq><variable id=\"TIRES_TYPE\" /><named-
constant id=\"SC 3\" /></eq>" },
            { expression: "<eq><variable id=\"TIRES_TYPE\" /><named-
constant id=\"SC 5\" /></eq>" },
            { expression: "<eq><variable id=\"TIRES_TYPE\" /><named-
constant id=\"PC 5\" /></eq>" },
          ]
        },
        "truth-table": [
          [1, 0, 0],
          [0, 1, 0],
          [0, 0, 1]
        ]
      }
    </Result>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>

```

5.2.7 Extra - How To

5.2.7.1 Get Unreachable Combinations

5.2.7.1.1 Overview

This section describes how to get a list of unreachable combinations. A combination is unreachable if it is valid in the current Scope but does not satisfy the specified expression. To resolve this task, use the `cfg_GetValidCombinations` method.

5.2.7.1.2 Prepare Expression

Task: Find unreachable combinations using the expression `<expression>e_1</expression>`.

Prepared expression: `<expression><not>e_1</not></expression>`

Task: Need to find unreachable combinations using a list of expressions:

```

<expression>e_1</expression>
<expression>e_2</expression>
<expression>e_3</expression>.

```

Prepared expression:

```

<expression>
  <not>
    <or>
      e_1

```

```
e_2  
e_3  
</or>  
</not>  
</expression>
```

A single expression

```
<expression>
...
</expression>
```

expression content is moving to

Unreachable combinations condition node should look like:

```
<condition>
  <![CDATA[
    <expression>
      <NOT>
        ...
      </NOT>
    </expression>
  ]]>
</condition>
```

Several separate expressions

```
<!-- expression #1 -->
<expression>
...
</expression>
<!-- expression #2 -->
<expression>
...
</expression>
...
<!-- expression #N -->
```

expression content is moving to

expression content is moving to

expression content is moving to

Unreachable combinations condition node should look like:

```
<condition>
  <![CDATA[
    <expression>
      <NOT>
        <OR>
          <!-- expression content #1 -->
          ...
          <!-- expression content #2 -->
          ...
          <!-- expression content #N -->
          ...
        </OR>
      </NOT>
    </expression>
  ]]>
```

<code><expression></code>	<code></NOT></code>
<code>...</code>	<code></expression></code>
<code></expression></code>	<code>]]></code>
	<code></condition></code>

5.2.7.1.3 Response Explanation

The `cfg_GetValidCombinations` method returns combinations that are not covered by any of the initial expressions. If the `cfg_GetValidCombinations` method does not return any combinations (an empty result), it means all valid combinations are “reachable” and are therefore covered by the list of initial expressions.

5.2.7.1.4 Examples

- Unreachable combinations exist

The Scope contains two variables: ‘Color’ and ‘Engine type’. The scope has no rules. You can set the ‘Color’ variable to either ‘Red’ or ‘Green’ values. You can set the ‘Engine type’ variable to either ‘Diesel’ or ‘Gasoline’ values. Since there are no rules, it means there are four available valid combinations: Red & Diesel, Red & Gasoline, Green & Diesel, Green & Gasoline.

You need to find all valid combinations that are not covered by the following expressions:

Verbal notation	Expression content
‘Color’ is ‘Red’	<pre> <EQ> <variable id="Color" /> <named-constant id="Red" /> </EQ> </pre>

The request:

```

<Item type="Method" action="cfg_GetValidCombinations" responseFormat="XML">
  <targetScope>
    <Item type="Method" action="..." id="..." />
  </targetScope>
  <condition>
    <![CDATA[
      <expression>
        <NOT>
          <EQ>
            <variable id="Color" />
            <named-constant id="Red" />
          </EQ>
        </NOT>
      </expression>
    ]]>
  </condition>
</Item>

```

The response contains two combinations: Green & Diesel and Green & Gasoline. Because they are not covered by the initial expression, they are referred to as unreachable combinations.

The XML response:

```
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Body>
    <Result>
      <combinations-meta>
        <variables>
          <variable id="Color" order="0" />
          <variable id="Engine type" order="1" />
        </variables>
        <values>
          <value type="NamedConstant" order="0">Green</value>
          <value type="NamedConstant" order="1">Diesel</value>
          <value type="NamedConstant" order="2">Gasoline</value>
        </values>
      </combinations-meta>
      <combinations>
        <combination>
          <value>0</value>
          <value>1</value>
        </combination>
        <combination>
          <value>0</value>
          <value>2</value>
        </combination>
      </combinations>
    </Result>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

The JSON response (if @responseFormat="JSON" is within the AML request):

```
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Body>
    <Result>
      {
        "combinations-meta": {
          "variables": {
            "Color": 0,
            "Engine type": 1
          },
          "values": [{
            "type": "NamedConstant",
            "id": "Green"
          },
          {
            "type": "NamedConstant",
            "id": "Diesel"
          },
          {
            "type": "NamedConstant",
            "id": "Gasoline"
          }
        ]
      },
    </Result>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

```

        "combinations": [[0,1],[0,2]]
    }
</Result>
</SOAP-ENV:Body>
</SOAP-ENV:Envelope>

```

- Unreachable combinations don't exist

The Scope contains two variables: 'Color' and 'Engine type'. The scope has no rules. You can set the 'Color' variable to either 'Red' or 'Green' values. You can set the 'Engine type' variable to either 'Diesel' or 'Gasoline' values. Since there are no rules, it means there are four available valid combinations: Red & Diesel, Red & Gasoline, Green & Diesel, Green & Gasoline.

You need to confirm that all valid combinations are covered by the following expressions:

Verbal notation	Expression content
'Color' is 'Red'	<pre> <EQ> <variable id="Color" /> <named-constant id="Red" /> </EQ> </pre>
'Color' is 'Green'	<pre> <EQ> <variable id="Color" /> <named-constant id="Green" /> </EQ> </pre>

The request is:

```

<Item type="Method" action="cfg_GetValidCombinations" responseFormat="XML">
  <targetScope>
    <Item type="Method" action="..." id="...">
      </targetScope>
      <condition>
        <![CDATA[
          <expression>
            <NOT>
              <OR>
                <EQ>
                  <variable id="Color" />
                  <named-constant id="Red" />
                </EQ>
                <EQ>
                  <variable id="Color" />
                  <named-constant id="Green" />
                </EQ>
              </OR>
            </NOT>
          </expression>
        ]]>
      </condition>
    </Item>

```

Since all valid combinations have the 'Color' set to 'Red' or 'Green', all valid combinations are reachable, and the response is empty.

The XML response:

```
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Body>
    <Result />
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

The JSON response (if @responseFormat="JSON" is within the AML request):

```
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Body>
    <Result>{}</Result>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

5.2.7.2 Get Unreachable Expressions

[5.2.7.2.1 Overview](#)

This section describes how to verify that an expression is unreachable. An expression is unreachable if there are no valid combinations in the current Scope using the specified expression. To resolve this task, use the `cfg_GetValidCombinations` method.

[5.2.7.2.2 Prepare Expression](#)

There is no specific format for the 'condition' node of the `cfg_GetValidCombinations` method request.

You must put your expression inside the 'condition' node. The following diagram shows how to create a proper 'condition' node.

An expression

```
<expression>  
...  
</expression>
```

expression is moving

Unreachable expression condition node should look like:

```
<condition>  
  <![CDATA[  
    <expression>  
    ...  
  </expression>  
  ]]>  
</condition>
```

5.2.7.2.3 Response Explanation

If the `cfg_GetValidCombinations` method response contains at least one valid combination, it means the specified expression is reachable. If the `cfg_GetValidCombinations` method response is empty, it means the specified expression is unreachable.

5.2.7.2.4 Examples

- Unreachable expression is detected

The Scope contains two variables: 'Color' and 'Engine type'. The scope has no rules. You can set the 'Color' variable to either 'Red' or 'Green' values. You can set the 'Engine type' variable to either 'Diesel' or 'Gasoline' values. Since there are no rules, it means there are four available valid combinations: Red & Diesel, Red & Gasoline, Green & Diesel, Green & Gasoline.

You need to verify that the following expression is reachable:

Verbal notation	Expression
	<expression>
'Color' is 'Red'	<AND> <EQ> <variable id="Color" /> <named-constant id="Red" />
'Color' is 'Green'	</EQ> <EQ>

Verbal notation	Expression
	<code><variable id="Color" /></code>
	<code><named-constant id="Green" /></code>
	<code></EQ></code>
	<code></AND></code>
	<code></expression></code>

The request:

```
<Item type="Method" action="cfg_GetValidCombinations" fetch="1" responseFormat="XML">
  <targetScope>
    <Item type="Method" action="..." id="..." />
  </targetScope>
  <condition>
    <![CDATA[
      <expression>
        <AND>
          <EQ>
            <variable id="Color" />
            <named-constant id="Red" />
          </EQ>
          <EQ>
            <variable id="Color" />
            <named-constant id="Green" />
          </EQ>
        </AND>
      </expression>
    ]]>
  </condition>
</Item>
```

The 'Color' variable cannot be set to 'Red' and 'Green' values at the same time. This means that the response will be empty. The passed expression is therefore an unreachable one because it doesn't lead to the availability of at least one valid combination.

The XML response:

```
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Body>
    <Result />
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

The JSON response (if @responseFormat="JSON" is contained within the AML request):

```
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Body>
    <Result>{}</Result>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

- Unreachable expression is not detected

The scope contains two variables: 'Color' and 'Engine type'. The scope has no rules. You can set the 'Color' variable to either 'Red' or 'Green' values. You can set the 'Engine type' variable to either 'Diesel' or 'Gasoline' values. Since there are no rules, it means there are four available valid combinations: Red & Diesel, Red & Gasoline, Green & Diesel, Green & Gasoline.

You need to verify that the following expression is reachable:

Verbal notation	Expression
	<code><expression></code>
	<code><OR></code>
	<code><EQ></code>
'Engine type' is 'Diesel'	<code><variable id="Engine type" /></code> <code><named-constant id="Diesel" /></code>
OR	<code></EQ></code>
	<code><EQ></code>
'Engine type' is 'Gasoline'	<code><variable id="Engine type" /></code> <code><named-constant id="Gasoline" /></code>
	<code></EQ></code>
	<code></OR></code>
	<code></expression></code>

The request:

```
<Item type="Method" action="cfg_GetValidCombinations" fetch="1" responseFormat="XML">
  <targetScope>
    <Item type="Method" action="..." id="..." />
  </targetScope>
  <condition>
    <![CDATA[
      <expression>
        <OR>
          <EQ>
            <variable id="Engine type" />
            <named-constant id="Diesel" />
          </EQ>
          <EQ>
            <variable id="Engine type" />
            <named-constant id="Gasoline" />
          </EQ>
        </OR>
      </expression>
    ]]>
  </condition>
</Item>
```

Since you can set the 'Engine type' variable to 'Diesel' or 'Gasoline' for a particular combination, the response will contain several valid combinations. This means that the passed expression is reachable because it leads to the availability of at least one valid combination.

The XML response:

```
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Body>
    <Result>
      <combinations-meta>
        <variables>
          <variable id="Color" order="0" />
          <variable id="Engine type" order="1" />
        </variables>
        <values>
          <value type="NamedConstant" order="0">Red</value>
          <value type="NamedConstant" order="1">Green</value>
          <value type="NamedConstant" order="2">Diesel</value>
          <value type="NamedConstant" order="3">Gasoline</value>
        </values>
      </combinations-meta>
      <combinations>
        <combination>
          <value>0</value>
          <value>1</value>
        </combination>
        <combination>
          <value>1</value>
          <value>2</value>
        </combination>
        <combination>
          <value>0</value>
          <value>3</value>
        </combination>
        <combination>
          <value>1</value>
          <value>3</value>
        </combination>
      </combinations>
    </Result>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

The JSON response (if @responseFormat="JSON" is contained within the AML request):

```
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Body>
    <Result>
      {
        "combinations-meta": {
          "variables": {
            "Color": 0,
            "Engine type": 1
          },
          "values": [{
            "type": "NamedConstant",
            "id": "Red"
```

```

    },
    {
      "type": "NamedConstant",
      "id": "Green"
    },
    {
      "type": "NamedConstant",
      "id": "Diesel"
    },
    {
      "type": "NamedConstant",
      "id": "Gasoline"
    }
  ]
},
"combinations": [[0,2],
[0,3],
[1,2],
[1,3]]
}
</Result>
</SOAP-ENV:Body>
</SOAP-ENV:Envelope>

```

Note: In order to verify the reachability of an expression, finding just a single valid combination is sufficient. It is recommended to use the “fetch=1” attribute for the AML request to `cfg_GetValidCombinations`.

5.3 Influence of Missing Terms on the API results

Some variables or their constants may not be found within the target scope during transformations. In these cases, the term Missing Term is used. Missing Term is a Term which contains at least one ID that is not in the target Scope. Any Missing Terms transform into a “False” constant value while solving a Scope.

- **Example**

The target Scope contains the “Color” variable, which contains “Red” and “Black” Named Constants:

```

<Item type="Scope" id="ScopeId">
  <name>ScopeName</name>
  <Relationships>
    <Item type="Variable" id="ColorVariableId">
      <id>ColorVariableId</id>
      <name>Color</name>
      <Relationships>
        <Item type="NamedConstant" id="NamedConstantId_1">
          <id>NamedConstantId_1</id>
          <name>Red</name>
        </Item>
        <Item type="NamedConstant" id="NamedConstantId_2">
          <id>NamedConstantId_2</id>
          <name>Black</name>
        </Item>
      </Relationships>
    </Item>
  </Relationships>
</Item>

```

```

    </Item>
  </Relationships>
</Item>

```

Consider the following expression concerning the target Scope:

```

<expression>
  <eq>
    <variable id="ColorVariableId" />
    <named-constant id="NamedConstantId_1" />
  </eq>
  <eq>
    <variable id="ColorVariableId" />
    <named-constant id="NamedConstantId_2" />
  </eq>
  <eq>
    <variable id="ColorVariableId" />
    <named-constant id="NamedConstantId_3" />
  </eq>
</expression>

```

In this case, the following term from the expression is considered as a Missing Term:

```

<eq><variable id="ColorVariableId" /><named-constant id="NamedConstantId_3" /></eq>

```

As result, the whole expression will be converted to "False".

6 Configurator Services Interface

It is possible to work with Configurator Services without using JSON/AML serialization. This is provided by a public interface `IConfiguratorServices` implemented in the `Aras.Server.Core.Abstractions` namespace.

The interface provides the following functionality described in section **Error! Reference source not found.** REF_Ref134710177 \h * MERGEFORMAT **Error! Reference source not found.**:

- alternate for the `cfg_GetScopeStructure` method (without `output_builder_method` parameter, see 5.2.1 `cfg_GetScopeStructure`);
- alternate for the `cfg_GetValidCombinations` method (see 5.2.2 `cfg_GetValidCombinations`)

`IConfiguratorServices` interface allows working with objects from Configurator Services without using JSON/AML. This can be beneficial in some cases from a performance perspective.

This section describes:

- Getting an `IConfiguratorServices` interface
- Interface description
- Usage example

6.1 Getting an `IConfiguratorServices` interface

`IConfiguratorService` interface object can be received from a call context in a server method.

```
CCO.GetService<Aras.Server.Core.Abstractions.IConfiguratorServices>();
```

6.2 Interface description

The interface definition is:

```
public interface IConfiguratorServices
{
    ReadOnlyScope GetScope(Item targetScopeItem);
    IScopeSolver CreateSolver(ReadOnlyScope scope);
}
```

Where:

<i>ReadOnlyScope</i>	Class representing a Scope object with list of rules and list of variables. It cannot be modified.
<i>IScopeSolver</i>	Interface that represents a scope solver. This interface is responsible for setting the scope and getting combinations.

Both interface methods are described below.

6.2.1 GetScope

```
public ReadOnlyScope GetScope(Item targetScopeItem);
```

Note: GetScope method provides similar functionality as `cfg_GetScopeStructure` (refer to 5.2.1 `cfg_GetScopeStructure`) except the `output_builder_method`.

Method returns `ReadOnlyScope` with immutable list of rules and list of variables. `ReadOnlyScope` class defined as:

```
public class ReadOnlyScope
{
    public string Id;
    public string Name;
    public ReadOnlyCollection<Rule> RuleList;
    public ReadOnlyCollection<Variable> VariableList;
}
```

Note: `RuleList` and `VariableList` contain references to the actual objects. Changing referenced Rules and Variables would change cached Scope Rules and Variables.

`targetScopeItem` parameter can be defined like (see 4.3.2)

```
Item targetScopeItem = this.newItem("Method", "%builder method name%");
targetScopeItem.setAttribute(
    "%builder method attribute name%",
    "%builder method attribute value%");
targetScopeItem.setProperty(
    "%builder method property name%",
    "%builder method property value%");
```

Where:

%builder method name%	Name of the server method that builds the Scope object model based on the business logic being used.
%builder method attribute name%	Builder method attribute name.
%builder method attribute value%	Builder method attribute value.
%builder method property name%	Builder method property name.
%builder method property value%	Builder method property value.

6.2.2 CreateSolver

```
public IScopeSolver CreateSolver(ReadOnlyScope scope);
```

`IScopeSolver` interface is declared as following:

```
public interface IScopeSolver
{
    ReadOnlyScope Scope { get; }
```

```

ExpressionBase Condition { get; set; }
IEnumerable<IEnumerable<ExpressionTerm>> FindAllCombinations(
    IList<Variable> selectVariables);
IEnumerable<IEnumerable<ExpressionTerm>> FindAllCombinations(
    IList<Variable> selectVariables,
    ExpressionBase additionalCondition);
}

```

Note: FindAllCombinations method provides similar functionality as cfg_GetValidCombinations (refer to 5.2.2 cfg_GetValidCombinations).

6.2.2.1 FindAllCombinations

Method returns all possible valid combinations. Returned object is a collection of `IEnumerable<ExpressionTerm>` objects. A single `IEnumerable<ExpressionTerm>` is a unique combination of all variables and their values.

ExpressionTerm class consists of two fields:

- OperandLeft – represents a certain variable in combination.
- OperandRight – represents a value (named-constant) corresponding to the variable.

Both operands contain an Id and Value of a variable (for the OperandLeft) and of named-constant (for the OperandRight).

6.3 Usage example

This section provides an example of using the public interface: There is a need to check if any valid combination will exist when certain variable options (values) are selected. Therefore, it needs to be checked if such selected variable options are valid in the Scope.

The corresponding server method code could be the following.

```

// Get IConfiguratorServices object.
Aras.Server.Core.Abstractions.IConfiguratorServices configuratorModule =
    CCO.GetService<Aras.Server.Core.Abstractions.IConfiguratorServices>();

// Set targetScope.
Item targetScopeItem = this.newItem("Method", "scopeBuilderMethod");
string businessItemId = "6B9B49B827C44F38A97BBFB524CA5BD5";
targetScopeItem.setID(businessItemId);

// Get scope.
Aras.Server.Core.Configurator.ReadOnlyScope scope =
    configuratorModule.GetScope(targetScopeItem);

// Create ScopeSolver.
Aras.Server.Core.Configurator.IScopeSolver scopeSolver =
    configuratorModule.CreateSolver(scope);

string selectedVariable1Id = "62419B0CF8B1464CA73FF72B072924E5";
string selectedOption1Id = "3021A89854F24F38918BCF2CFC27CAD3";
string selectedVariable2Id = "0C6562E7925C4215ABE6E31EF0ED1BD2";

```

```

string selectedOption2Id = "B574E44C807545C2B74BA8A30CA8B319";

// Create selected variable options as a condition and set them to the solver.
scopeSolver.Condition = new Aras.Server.Core.Configurator.ExpressionLogical(
    Aras.Server.Core.Configurator.ExpressionType.AND);
scopeSolver.Condition.InnerExpressions.AddRange(
    new List<Aras.Server.Core.Configurator.ExpressionBase>
    {
        // SelectedVariableId is the Id of the selected variable.
        // SelectedOptionId is the Id of corresponding option(value) of the selected variable.
        new Aras.Server.Core.Configurator.ExpressionTerm(
            Aras.Server.Core.Configurator.TermOperator.Equal,
            new Aras.Server.Core.Configurator.TermOperandVariable(selectedVariable1Id),
            new Aras.Server.Core.Configurator.TermOperandNamedConstant(selectedOption1Id)),
        new Aras.Server.Core.Configurator.ExpressionTerm(
            Aras.Server.Core.Configurator.TermOperator.Equal,
            new Aras.Server.Core.Configurator.TermOperandVariable(selectedVariable2Id),
            new Aras.Server.Core.Configurator.TermOperandNamedConstant(selectedOption2Id)),
    });

// Null parameter for FindAllCombinations means that all scope variables will be used.
return scopeSolver.FindAllCombinations(null).Any();

```

7 Control API

There are two UI Controls available in Configurator Services: RuleEditor and VariantsTree. The following sections describe the APIs for each.

7.1 RuleEditor

The RuleEditor control enables users to input text using an “input group template”. You can either enter text manually or by using the intellisense menu. You can also customize the view for individual groups or within a group for specific group types.

7.1.1 Constructor

The Constructor supports one composite parameter. The following table describes its properties.

Name	Type	Description
connectId	String	Mandatory parameter. It is the ID of the DOM element container for VariantsTree control. This element should exist in the DOM document. The VariantsTree DOM is placed as a child into the container. The default is ‘variantsTreeContainer’.
contextMenuEnabled	Boolean	Optional parameter. Turns context menu support on/off (widget creation, events tracking). The default is False.
isEditable	Boolean	Optional parameter. Turns text editing for the control on/off. The default is False.

Name	Type	Description
template	Object	Optional composite parameter. It must include a 'template' array property containing the group names sequence and the parameters for each group from 'template' keyed with group names. Example: <pre>{ template: ['RuleConditionGroup'], templateGroups: { 'RuleConditionGroup': { type: 'grammar', grammarFile: 'RuleEditor/RuleGrammar.txt', title: 'Rule condition expression', }, } }</pre>

7.1.2 Public Fields

Name	Type	Description
containerNode	DOM Node	A reference to the DOM Node used as a container for the control DOM.
domNode	DOM Node	A reference to the DOM Node that corresponds to the control.
contextMenu	Object, MenuWidget	A reference to the menu widget used for the context menu.
intelliSenseMenu	Object, MenuWidget	A reference to the menu widget used for the intellisense menu.

7.1.3 Public Methods

This section describes the public methods.

7.1.3.1 *setInputTemplate*

Use this method to create a new editor input template. The startNewInput method needs to be called before you can start working with the editor using this template method.

Input parameters

Name	Type	Description
inputTemplate	Object	A template that contains a list of groups used as the 'template' array property as well as settings for each group.

7.1.3.2 *getInputTemplate*

Use this method to get the current editor input template.

Return value

Object. Template descriptor or null.

7.1.3.3 *setEditState*

Switch edit state for the control.

Input parameters

Name	Type	Description
isEditable	Boolean	The applied value defines how the editor content can be modified.

7.1.3.4 *isEditable*

Get edit state of the control. It determines whether content is editable.

Return value

Boolean

7.1.3.5 toggleCssClasses

This method enables toggling between DOM Node CSS classes.

Input parameters

Name	Type	Description
cssClassNames	String Array of Strings	Class names that can be added or removed from the Node. You can pass several class names as a string parameter by separating them using the 'space' character.
turnStyleOn	Boolean	If true, classes are added to the Node. If false, classes are removed.
targetNode	DOM Node, Nullable	Node that can be modified.

7.1.3.6 isInputValid

Determines if the values from all the input groups passed validation.

Return value

Boolean

7.1.3.7 isInputStarted

Determines if the template was set and all input groups were created.

Return value

Boolean

7.1.3.8 focus

Places focus into the active or first input group.

7.1.3.9 focusgroup

Places focus into a specific input group.

Input parameters

Name	Type	Description
groupName	String	Group name from template.

7.1.3.10 *focusFirstEditableGroup*

Places focus on the first input group, which supports user input.

7.1.3.11 *undoChange*

Undoes the previous editor value change.

7.1.3.12 *redoChange*

Re-does the change that was undone.

7.1.3.13 *getCurrentState*

Gets information about the current editor state.

Return value

Object with editor state information.

7.1.3.14 *resetChanges*

Resets all changes. The Editor returns to its initial values.

7.1.3.15 *setStartState*

Sets up the editor according to state settings. The current changes queue is dropped.

Input parameters

Name	Type	Description
startState	Object	State information for editor settings restoration.

7.1.3.16 *getValue*

Gets the current editor value.

Return value

Object with values from all input groups. The key is the group name from the template.

7.1.3.17 *getStringValue*

Gets the editor value as string.

Return value

String. Concatenation of all input group values.

7.1.3.18 setValue

Sets the editor value.

Input parameters

Name	Type	Description
inputData	Object	Value collection for editor input groups. Key = group name from template.

7.1.3.19 getErrorString

Gets value validation error as a string.

Return value

String. Concatenation of validation errors from all input groups.

7.1.3.20 isEditorFocused

Determines which editor control has focus.

Return value

Boolean

7.1.3.21 isMenuFocused

Determines which target menu control has focus.

Input parameters

Name	Type	Description
targetMenu	Object, MenuWidget	Menu widget used in the editor.

Return value

Boolean

7.1.3.22 isValueModified

Identifies whether the initial editor value was modified.

Return value

Boolean

7.1.3.23 registerShortcut

Registers the shortcut for the editor.

Input parameters

Name	Type	Description
keyCode	Number	Button key code.
isCtrl	Boolean	Specifies whether the Ctrl key is pressed.
isShift	Boolean	Specifies whether the Shift key is pressed.
callbackHandler	Function	Method that is called when the shortcut is activated.

7.1.3.24 getGroupIndex

Gets the index for a particular input group.

Input parameters

Name	Type	Description
targetGroup	Object, InputGroup	Registered editor input group.

7.1.3.25 getGroupByIndex

Gets a particular input group by index.

Input parameters

Name	Type	Description
groupIndex	Number	Group index from template.

Return value

InputGroup

7.1.3.26 *getGroupByDomNode*

Gets input group using the related DOM Node.

Input parameters

Name	Type	Description
targetDomNode	DOM Node	DOM Node belonging to the registered editor input group.

Return value

InputGroup

7.1.3.27 *getGroupName*

Gets the name of a specific input group.

Input parameters

Name	Type	Description
targetGroup	Object, InputGroup	Registered editor input group.

Return value

String. Group name from template.

7.1.3.28 *getGroupByName*

Gets registered editor input group by name from the template.

Input parameters

Name	Type	Description
groupName	String	Group name from active editor template.

Return value

InputGroup

7.1.3.29 *getGroupsCount*

Counts the editor input groups.

Return value

Number

7.1.3.30 *getNextGroup*

Gets the next input group relative to the target group. If the target group is last, then the first group is returned.

Input parameters

Name	Type	Description
targetGroup	Object, InputGroup	Group relative to which next group is searched.

Return value

InputGroup

7.1.3.31 *getPrevGroup*

Gets the previous input group relative to the target group. If the target group is last, then the first group is returned.

Input parameters

Name	Type	Description
targetGroup	Object, InputGroup	Group relatively to which previous group is searched.

Return value

InputGroup

7.1.3.32 *getGroupDomNode*

Gets the previous input group relative to the target group. If target group is last, then the first group is returned.

Input parameters

Name	Type	Description
targetGroup	Object, InputGroup String	Existing editor input group or it id.

Return value

InputGroup

7.1.3.33 startNewInput

Creates editor input groups based on the current template. All previous input groups are removed.

7.1.3.34 showIntelliSense

Shows the intellisense menu for a particular input group.

Input parameters

Name	Type	Description
targetGroup	Object, InputGroup	Existing editor input group.
allOptions	Boolean	If true, all available group values appear. If false, proposed values are limited based on the current group value.

7.1.3.35 hideIntelliSenseMenu

Hides active intellisense menu.

7.1.4 Events

Events described in this section are based on the Eventable core module. You can assign Handlers for these events using the **addEventListener** method.

7.1.4.1 onFocus

Event fires when the editor control has focus.

7.1.4.2 onBlur

Event fires when the focus moves away from the editor control.

7.1.4.3 *onStateChanged*

Event fires after group values are changed or the group loses focus.

7.1.4.4 *onRender*

Event fires after the editor creates DOM for input groups.

7.1.4.5 *onValueChanged*

Event fires after editor creates DOM for input groups.

7.1.4.6 *onGroupValueEntered*

Event fires if the group value changed and passed validation.

Input parameters

Name	Type	Description
targetGroup	Object, InputGroup	Affected editor input group.
approvedValue	String	Applied value, which passed validation.

7.1.5 Context Menu Events

The events described in this section belong to the dojo context menu widget. Use the dojo.connect and dojo.on modules to attach handlers for them.

7.1.5.1.1 *onMenuInit*

Event raised before context menu creation. Use it to create a list of required menu items.

Input parameters

Name	Type	Description
menuEvent	Event	Corresponding native context menu event.
targetGroup	Object, InputGroup	Target editor input group that the context menu is created for.
targetGroupName	String	Input group name from template.

Return value

Array of menu items descriptors.

7.1.5.1.2 onMenuItemClick

Event raised when the user clicks on the editor context menu item.

Input parameters

Name	Type	Description
commandId	String	ID of selected menu item.
contextData	Any	Context information for menu.

Return value

Array of menu items descriptors.

7.2 VariantsTree

The VariantsTree Control enables users to create and interact with a layered view. The view contains the following default layers:

- TreeVisualization
- GroupVisualization

The view is scalable. Context menus are supported.

7.2.1 Constructor

The Constructor supports one composite parameter that can contain the properties described in the following table:

Name	Type	Description
connectId	String	Mandatory property. It contains the DOM element container ID for the VariantsTree control. The Element should exist in the document. The VariantsTree DOM will be placed as a child into container. The default is 'variantsTreeContainer'.

Name	Type	Description
layerVisualizationParameters	Object	Optional composite parameter. It contains a collection of additional visual settings for view layers, which can be loaded into view. Collection key = layer id. Example: <pre>{ 'groups': { labelWidth: 200 } }</pre>

7.2.2 Public Fields

Name	Type	Description
layeredView	Object, MultiLayeredView	Reference to MultiLayeredView control. Most parts of the VariantsTree functionality are based on this core control. It is initialized in the constructor.

7.2.3 Public Methods

This section describes the public methods.

7.2.3.1 loadTree

The loadtree method creates TreeLayer and GroupLayer instances, loads them into the control and then centers the view. The TreeLayer instance is initialized with the treeData input parameter. TreeLayer gets the 'variantsTree' ID and GroupLayer gets the 'groups' ID. The Method can be overridden in order to change the list of loaded layers.

Input parameters

Name	Type	Description
treeData	Object	Hierarchical object data structure with a single root, where the next level children are placed into property children.

7.2.3.2 *isTreeLoaded*

This method verifies that the default view layers are already created and loaded into the view control (loadTree method was called).

Return value

Boolean

7.2.3.3 *getParentNode*

Returns the SVG Node related to the control.

Return value

DOM Node

7.2.3.4 *setGroupsLayerVisibility*

Allows the Groups layer to be either shown or hidden.

Input parameters

Name	Type	Description
isVisible	Boolean	Required layer visibility state.

7.2.3.5 *setTreeLayerData*

Sets new data for the layer.

Input parameters

Name	Type	Description
layerId	String Number	Layer id or index, which should be updated.
newData	Any	New layer data.

7.2.3.6 *getLeafDataNodesCount*

Gets count of leaf data items from TreeLayer.

Return value

Number of leafs.

7.2.3.7 *setScale*

Gets count of leaf data items from TreeLayer.

Input parameters

Name	Type	Description
newScale	Number	Sets new scale value for layered view control.

7.2.3.8 *getScale*

Get count of leaf data items from TreeLayer.

Return value

Number

7.2.3.9 *setSpacing*

Sets new spacing value for Tree layer. This parameter effects distance between rendered item nodes, which are on adjacent hierarchy levels (horizontal distance).

Input parameters

Name	Type	Description
spacingValue	Number	Spacing distance in pixels

7.2.3.10 *centerView*

Calculates and sets the optimal view scale when layers content fits client viewport.

7.2.3.11 *adjustViewBounds*

Recalculates SVG node size based on current view content bounds and scale. This is a complex operation, which should be used after scaling or significant content changes.

7.2.4 Events

The following sections describe the events that can be used as part of the VariantsTree Control.

7.2.4.1 *onScaleChanged*

Event fires when view scale is changed.

Input parameters

Name	Type	Description
newScale	Number	Scale value that was applied.

7.2.4.2 onGroupsSwitched

Event fires when view scale is changed.

Input parameters

Name	Type	Description
newScale	Number	Scale value that was applied.

7.2.5 Context Menu Events

The events described in this section belong to the dojo context menu widget. Use the dojo.connect and dojo.on modules to attach handlers for them.

7.2.5.1 onTreeMenuInit

Event handler for onMenuInit, which generates array of descriptors for all available context menu actions.

Return value

Array of menu items descriptors.

7.2.5.2 onTreeMenuSetup

Event handler for setup context menu item's state based on target layer and node.

Input parameters

Name	Type	Description
menuControl	Object, MenuWidget	Menu control, which should be updated.
targetLayer	Object, VisualizationLayer	Layer that contains Node for which menu should be displayed.

Name	Type	Description
targetDataNode	Object	Data object that bound to the target DOM Node.

7.2.5.3 onMenuItemClick

Handler for onMenuItemClick event.

Input parameters

Name	Type	Description
commandId	String	Menu item id.
targetDataNode	Object	Data object that is bounded to the target DOM Node.